

ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC BÁCH KHOA
KHOA KHOA HỌC VÀ KỸ THUẬT MÁY TÍNH



BÀI TẬP LỚN
**PHÁT TRIỂN ỨNG DỤNG INTERNET OF
THINGS (CO3037)**

Internet of Things Project Assignment

Giảng viên hướng dẫn: Lê Trọng Nhân
HK252

STT	Họ và tên	MSSV	Lớp	Ghi chú
1	Hồ Đăng Khoa	2211588	L01	Nhóm trưởng
2	Hoàng Sỹ Xuân Sơn	2212937	L01	
3	Vũ Đức Lâm	2212466	L01	
4	Nguyễn Minh Hưng	2211367	L01	

Đường dẫn mã nguồn dự án (GitHub):

https://github.com/username/repository_name



Thành phố Hồ Chí Minh, tháng 5 năm 2026

Danh sách thành viên

STT	Họ và tên	MSSV	Nhiệm vụ	Đóng góp
1	Hồ Đăng Khoa	2211588	Nhóm trưởng, thiết kế kiến trúc đa nhiệm FreeRTOS, hiện thực Task LED đơn, Task NeoPixel và tích hợp các cơ chế đồng bộ hóa Mutex/Semaphore.	100%
2	Hoàng Sỹ Xuân Sơn	2212937	Nghiên cứu sửa lỗi kết nối MQTT CoreIOT, thiết kế chế độ nhấp nháy LED, tham gia kiểm thử tích hợp hệ thống và viết báo cáo.	100%
3	Vũ Đức Lâm	2212466	Hiện thực Task đo đặc cảm biến DHT20 và hiển thị LCD 1602 giao tiếp I2C bảo vệ bằng Mutex, tham gia quay video thực nghiệm và viết báo cáo.	100%
4	Nguyễn Minh Hưng	2211367	Huấn luyện mô hình TinyML phân loại bất thường, tích hợp TensorFlow Lite Micro với Tensor Arena 8KB tĩnh, thiết lập Webserver, WebSocket và quản lý cấu hình qua LittleFS.	100 %

Mục lục

1	Giới thiệu	4
1.1	Lý do chọn đề tài	4
1.2	Bối cảnh ứng dụng thực tế	4
1.3	Mục tiêu của hệ thống	5
1.4	Phạm vi và giới hạn	5
2	Cơ sở lý thuyết	7
2.1	Phần cứng	7
2.1.1	Bo mạch lập trình nhúng Yolo UNO và Vi điều khiển ESP32-S3	7
2.1.2	Cảm biến nhiệt độ và độ ẩm DHT20	8
2.1.3	Đèn LED đơn cảnh báo (Tích hợp onboard)	10
2.1.4	Đèn LED màu NeoPixel WS2812B (Tích hợp onboard)	10
2.1.5	Màn hình hiển thị LCD I2C	11
2.1.6	Giao thức truyền thông và kết nối	13
2.2	FreeRTOS	13
2.2.1	Quản lý tác vụ (Task Management)	13
2.2.2	Cơ chế lập lịch thời gian thực	14
2.2.3	Giao tiếp và đồng bộ giữa các task	14
2.2.4	Quản lý bộ nhớ	14
2.2.5	Software Timer	15
2.2.6	Tiết kiệm năng lượng	15
2.2.7	Khả năng mở rộng và tương thích	15
2.2.8	Các thư viện hỗ trợ IoT	16
2.3	CoreIOT	16
2.3.1	Các chức năng chính của CoreIOT	16
2.3.2	Kiến trúc hệ thống	16
2.3.3	Đặc điểm nổi bật	17
2.4	TinyML	17
2.4.1	Đặc điểm của TinyML	17

2.4.2	Kiến trúc hoạt động của TinyML	17
2.4.3	TensorFlow Lite Micro	18
2.4.4	Ứng dụng của TinyML trong IoT	18
3	Thiết kế hệ thống	19
3.1	Tổng quan hệ thống	19
3.2	Khối hiển thị LED, NeoPixel và LCD (Cảnh báo cục bộ)	21
3.3	Khối Webserver + Điều khiển cục bộ (Task 4)	22
3.4	Khối CoreIOT + Kết nối mạng STA (Task 6)	24
3.5	Khối TinyML (Task 5)	24
3.6	Khối quản lý cấu hình và Định danh thiết bị (LittleFS)	27
4	Hiện thực hệ thống	30
4.1	Kiến trúc Đa nhiệm FreeRTOS trên Yolo UNO	30
4.2	Hiện thực Khối cảnh báo và hiển thị ngoại vi	31
4.2.1	Tác vụ điều khiển LED đơn (Task 1)	31
4.2.2	Tác vụ điều khiển NeoPixel (Task 2)	32
4.2.3	Tác vụ đo đặc cảm biến và hiển thị LCD (Task 3)	33
4.3	Hiện thực Khối Webserver và giao thức WebSocket	35
4.4	Hiện thực Khối kết nối đám mây CoreIOT và mDNS	37
4.5	Hiện thực Khối TinyML và Tự kiểm thử (Startup Test)	39
4.5.1	Kịch bản tự kiểm thử khi khởi động (Startup Test)	39
4.5.2	Tác vụ suy luận thời gian thực TinyML (Task 5)	40
4.6	Hiện thực Khối quản lý cấu hình động bằng LittleFS	41
5	Kết quả thực nghiệm	43
5.1	Webserver và Cảnh báo LED cục bộ	43
5.2	Đồng bộ dải màu LED NeoPixel và hiển thị LCD	43
5.3	Đẩy dữ liệu lên đám mây CoreIOT (MQTT)	44
5.4	Mô hình TinyML và Tự kiểm thử (Startup Test)	45
6	Kết luận	47



6.1	Những kết quả đạt được	47
6.2	Hạn chế và Hướng phát triển	48
6.3	Đường dẫn mã nguồn dự án (GitHub)	49

1 Giới thiệu

1.1 Lý do chọn đề tài

Trong sự phát triển không ngừng của Internet vạn vật (IoT), việc xây dựng các hệ thống nhúng đòi hỏi khả năng xử lý nhiều tác vụ đồng thời với độ trễ thấp và độ chính xác cao. Để đáp ứng yêu cầu khắt khe này, Hệ điều hành thời gian thực (RTOS) trở thành một công cụ không thể thiếu đối với các kỹ sư và lập trình viên. Việc ứng dụng RTOS giúp tối ưu hóa tài nguyên phần cứng, đồng thời quản lý hiệu quả các luồng dữ liệu liên tục từ cảm biến.

Xuất phát từ nhu cầu nắm bắt các kỹ thuật lập trình nhúng nâng cao, nhóm đã quyết định thực hiện đề tài dựa trên việc sửa đổi và mở rộng dự án nền tảng YoloUNO_PlatformIO - RTOS_Project. Đề tài mang lại cơ hội thực hành trực tiếp các kỹ thuật đồng bộ hóa tác vụ bằng tín hiệu (signals) và semaphore, loại bỏ hoàn toàn việc sử dụng các biến toàn cục thiếu an toàn. Đồng thời, dự án còn tích hợp các công nghệ hiện đại như Trí tuệ nhân tạo tại biên (TinyML) và giao tiếp đám mây (Cloud Server), tạo tiền đề vững chắc cho việc phát triển các sản phẩm IoT phức tạp trong thực tế.

1.2 Bối cảnh ứng dụng thực tế

Các hệ thống nhúng hiện đại không chỉ dừng lại ở việc thu thập dữ liệu đơn thuần mà đang tiến tới khả năng "tự suy nghĩ" và phân tích dữ liệu ngay tại nơi thu thập (Edge Computing). Trong bối cảnh đó, sự kết hợp giữa RTOS và TinyML mở ra khả năng tạo ra các thiết bị thông minh có thể nhận diện môi trường, dự đoán sự cố và phản hồi tức thì mà không cần phụ thuộc hoàn toàn vào máy chủ trung tâm.

Mô hình dự án này phản ánh trực tiếp cấu trúc của một hệ thống IoT công nghiệp cỡ nhỏ: từ lớp cảm nhận (thu thập nhiệt độ, độ ẩm), lớp xử lý cục bộ (giao diện Web AP, xử lý TinyML, cảnh báo qua LCD/LED), cho đến lớp mạng và đám mây (truyền dữ liệu lên nền tảng CoreIOT). Những kiến thức và kỹ năng từ dự án này có thể áp dụng trực tiếp vào việc thiết kế các hệ thống nhà thông minh (Smart Home), trạm quan trắc nông nghiệp, hoặc thiết bị y tế đeo tay.

1.3 Mục tiêu của hệ thống

Mục tiêu chính của đề tài là xây dựng một hệ thống giám sát và điều khiển thông minh trên vi điều khiển ESP32 S3, đảm bảo mã nguồn mới phải có chức năng khác biệt ít nhất 30% so với mã nguồn gốc. Để đạt được điều này, hệ thống cần hoàn thành các mục tiêu cụ thể sau:

- **Cải tiến kỹ thuật nhúng:** Quản lý trạng thái nhấp nháy của đèn LED theo nhiệt độ và cấu hình màu sắc của LED NeoPixel dựa trên độ ẩm (ít nhất 3 mức độ khác nhau) thông qua semaphore và tín hiệu đồng bộ.
- **Quản lý hiển thị:** Sử dụng semaphore để quản lý tài nguyên, hiển thị thông số và ít nhất 3 trạng thái cảnh báo trên màn hình LCD mà không dùng biến toàn cục.
- **Giao diện tương tác:** Thiết lập ESP32 S3 làm điểm truy cập (Access Point) và thiết kế một máy chủ web trực quan cho phép người dùng điều khiển ít nhất hai thiết bị ngoại vi.
- **Triển khai TinyML:** Xây dựng bộ dữ liệu, huấn luyện, chạy mô hình TinyML trên phần cứng và tiến hành đo lường độ chính xác.
- **Tích hợp đám mây:** Cấu hình ESP32 S3 ở chế độ Trạm (STA) để xuất bản thành công dữ liệu cảm biến lên máy chủ đám mây CoreIOT bằng mã xác thực hợp lệ.

1.4 Phạm vi và giới hạn

Dự án được triển khai toàn bộ trên nền tảng vi điều khiển ESP32 S3 bằng ngôn ngữ C/C++ thông qua plugin PlatformIO. Trong giới hạn của đề tài, cấu hình phần cứng của bo mạch được cố định ở chế độ 301B9 và 812H6.

Về mặt dữ liệu và kết nối, hệ thống giới hạn việc thu thập các thông số cơ bản (nhiệt độ, độ ẩm) và truyền dữ liệu thông qua giao thức Wi-Fi hiện có tại phòng thí nghiệm. Quá trình huấn luyện mô hình TinyML sử dụng tập dữ liệu có quy mô nhỏ gọn, phù



hợp với dung lượng bộ nhớ của ESP32 S3, và nền tảng CoreIOT được sử dụng như giải pháp lưu trữ đám mây duy nhất thay vì triển khai một máy chủ (Server) độc lập. Việc kiểm thử hệ thống được thực hiện trong môi trường mô phỏng (phòng Lab), tập trung vào tính logic của thuật toán quản lý RTOS thay vì tối ưu hóa cho điều kiện hoạt động khắc nghiệt ngoài trời.

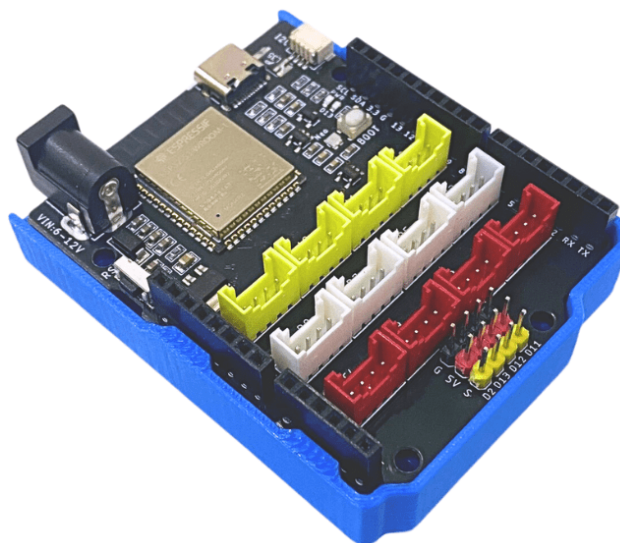
2 Cơ sở lý thuyết

2.1 Phần cứng

2.1.1 Bo mạch lập trình nhúng Yolo UNO và Vi điều khiển ESP32-S3

Yolo UNO là một board mạch lập trình nhúng được thiết kế và phát triển nhằm phục vụ việc nghiên cứu, thử nghiệm và triển khai các ứng dụng kết nối vạn vật IoT và Trí tuệ nhân tạo (AI). Board mạch được xây dựng trên nền tảng vi điều khiển SoC dòng ESP32-S3 mã hiệu ESP32-S3-WROOM-1, sở hữu bộ xử lý lõi kép Xtensa 32-bit với tốc độ xung nhịp lên tới 240MHz. Điểm nổi bật của cấu hình phần cứng này là việc tích hợp sẵn 16MB bộ nhớ Flash và 8MB bộ nhớ PSRAM mở rộng, mang lại không gian lưu trữ và xử lý mượt mà cho các ứng dụng đa nhiệm thời gian thực và nạp các mô hình học máy tại biên (TinyML).

Bo mạch hỗ trợ đồng thời hai chế độ kết nối không dây 2.4GHz Wi-Fi (chuẩn 802.11 b/g/n) và Bluetooth 5 (BLE), tích hợp sẵn 12 cổng kết nối chuẩn Grove mở rộng phân tách rõ ràng thành các cổng Analog, Digital và I2C, cho phép thiết bị dễ dàng tương tác với hệ sinh thái linh kiện cảm biến ngoại vi cận biên. Trong dự án này, bo mạch được cấu hình hoạt động đồng bộ ở các chế độ phần cứng chuyên dụng 301B9 và 812H6.



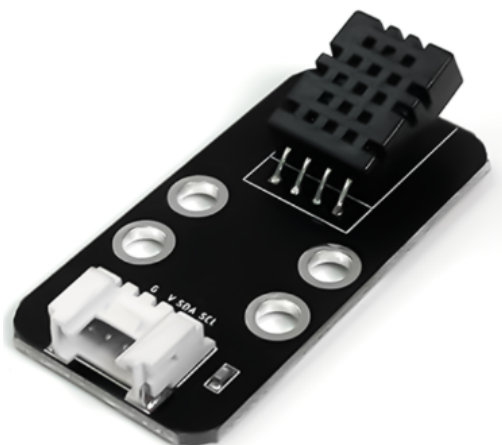
Hình 1: Board lập trình Yolo UNO và chip xử lý ESP32-S3

Bảng 1: Thông số kỹ thuật của board lập trình Yolo UNO trong hệ thống

Thông số	Chi tiết
Vi xử lý	ESP32-S3 Dual Core 240MHz Tensilica Xtensa
Bộ nhớ mở rộng	16MB Flash, 8MB PSRAM
Kết nối không dây	2.4GHz Wi-Fi (802.11 b/g/n), Bluetooth 5.0, BLE
Nguồn cấp hoạt động	5VDC qua cổng USB Type-C hoặc Jack DC 5521
Nút nhấn tích hợp	Phím Reset, Phím chế độ Bootloader (BOOT0)
Giao tiếp mở rộng	12 cổng Grove (4 × Analog, 4 × Digital, 4 × I2C)
Đèn hiển thị onboard	LED nguồn, LED chân D13, bóng LED RGB NeoPixel
Chế độ cấu hình	Thiết lập chuẩn phần cứng 301B9 và 812H6

2.1.2 Cảm biến nhiệt độ và độ ẩm DHT20

DHT20 là dòng cảm biến nhiệt độ và độ ẩm kỹ thuật số thế hệ mới, được nâng cấp đáng kể so với các thế hệ trước. Cảm biến tích hợp một chip ASIC chuyên dụng bên trong, cùng với một phần tử cảm biến độ ẩm điện dung (capacitive humidity sensor) và một cảm biến nhiệt độ tiêu chuẩn. Đặc điểm nổi bật nhất của DHT20 là việc hỗ trợ giao thức truyền thông I2C (Inter-Integrated Circuit), giúp quá trình giao tiếp với vi điều khiển ESP32-S3 trở nên ổn định, chống nhiễu tốt hơn và dễ dàng tích hợp vào hệ thống bus chung. Mỗi cảm biến DHT20 đều được hiệu chuẩn nghiêm ngặt trong phòng thí nghiệm trước khi xuất xưởng, đảm bảo độ chính xác và tính đồng nhất cao cho các ứng dụng IoT giám sát môi trường thời gian thực.



Hình 2: Cảm biến nhiệt độ và độ ẩm kỹ thuật số I2C DHT20

Bảng 2: Thông số kỹ thuật của cảm biến nhiệt độ và độ ẩm DHT20

Thông số	Giá trị
Điện áp hoạt động	2.2V – 5.5V DC
Giao thức truyền dữ liệu	I2C (Inter-Integrated Circuit)
Phạm vi đo độ ẩm	0% – 100% RH
Phạm vi đo nhiệt độ	-40°C – 80°C
Độ chính xác độ ẩm	±3% RH
Độ chính xác nhiệt độ	±0.5°C

Bảng 3: Chức năng các chân kết nối của cảm biến DHT20

STT	Chân	Chức năng
1	VDD	Cấp nguồn dương (2.2V – 5.5V)
2	SDA	Dây truyền nhận dữ liệu nối tiếp (Serial Data)
3	GND	Nối đất
4	SCL	Dây cấp xung nhịp nối tiếp (Serial Clock)

2.1.3 Đèn LED đơn cảnh báo (Tích hợp onboard)

Đèn LED đơn là một linh kiện bán dẫn phát quang hoạt động dựa trên hiện tượng giải phóng năng lượng dưới dạng photon khi có dòng điện thuận chạy qua tiếp giáp p-n. Đóng vai trò là thiết bị chỉ thị cục bộ trực quan, trong kiến trúc của hệ thống này, đèn LED đơn không phải là một module rời mà được **tích hợp sẵn (onboard) ngay trên mạch Yolo UNO** (kết nối trực tiếp với chân GPIO 38/D13 của vi điều khiển ESP32-S3). Do đèn LED được tích hợp sẵn trực tiếp trên bo mạch chính Yolo UNO, hệ thống không sử dụng module LED đơn ngoại vi riêng lẻ, do đó không có hình ảnh minh họa đấu nối phần cứng độc lập cho linh kiện này. Việc sử dụng linh kiện onboard giúp tiết kiệm không gian lắp ráp và chân cắm phần cứng. Dưới sự điều phối của FreeRTOS và cơ chế Semaphore, chu kỳ đóng/ngắt dòng điện cấp vào LED được biến đổi tuần tự để tạo ra ít nhất 3 trạng thái tần số nhấp nháy khác nhau phản ánh mức độ khẩn cấp của nhiệt độ môi trường.

Bảng 4: Thông số kỹ thuật tiêu chuẩn của LED đơn cảnh báo

Thông số	Giá trị
Điện áp kích hoạt thuận	1.8V – 2.2VDC (tùy thuộc màu sắc LED)
Dòng điện hoạt động định mức	10mA – 20mA
Dạng tín hiệu điều khiển	Kích hoạt mức logic (High/Low) hoặc xung PWM

2.1.4 Đèn LED màu NeoPixel WS2812B (Tích hợp onboard)

NeoPixel là một giải pháp LED màu RGB thông minh tích hợp sẵn chip vi điều khiển IC dòng WS2812B ngay bên trong khoang bóng LED. Cấu trúc này cho phép điều khiển độc lập cường độ sáng và màu sắc của từng pixel riêng biệt thông qua một dây tín hiệu số duy nhất sử dụng giao thức truyền thông định thời dòng đơn mã RZ (Return-to-Zero). Mỗi bóng LED yêu cầu một chuỗi dữ liệu 24-bit màu (8-bit cho dải Xanh lá, 8-bit cho dải Đỏ, và 8-bit cho dải Xanh dương). Tương tự như LED đơn, bóng NeoPixel cũng được **tích hợp sẵn (onboard) trực tiếp trên bo mạch Yolo UNO**. Việc ứng dụng bóng NeoPixel onboard này giúp hệ thống hiển thị linh hoạt ít

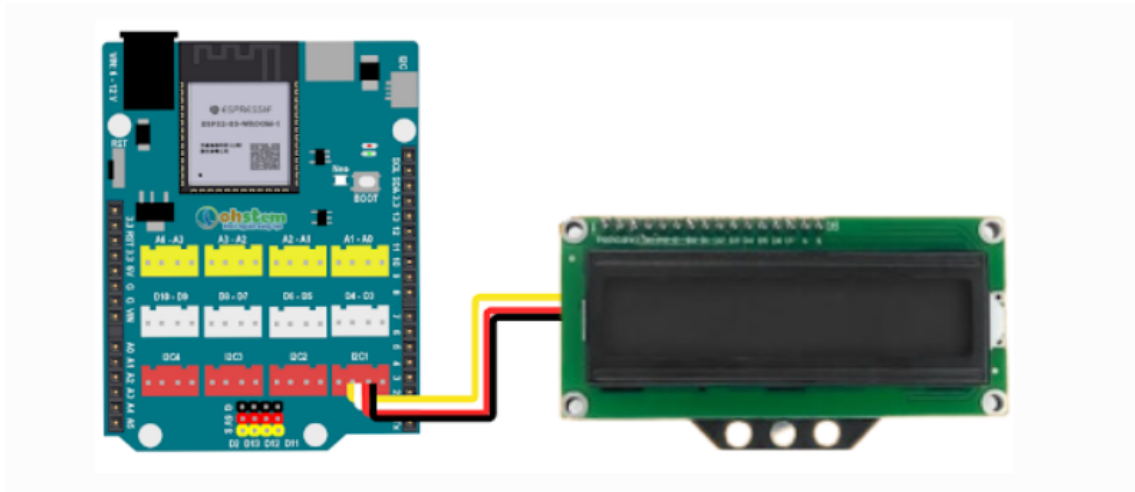
nhất 3 mẫu màu sắc cấu hình để cảnh báo trực quan tương quan chính xác theo sự biến động của dải độ ẩm môi trường cận biên.

Bảng 5: Thông số kỹ thuật của bóng LED màu NeoPixel onboard

Thông số	Giá trị
Điện áp hoạt động cấp vào	5VDC
Kiểu IC điều khiển tích hợp	WS2812B
Định dạng cấu trúc màu sắc	RGB 24-bit (16,7 triệu màu sắc)
Tần số truyền nhận dữ liệu số	800 Khz

2.1.5 Màn hình hiển thị LCD I2C

Màn hình hiển thị tinh thể lỏng LCD (chuẩn ký tự dạng 16x2 hoặc 20x4) là linh kiện hiển thị văn bản cục bộ dùng để xuất ra trực tiếp các thông số kỹ thuật của hệ thống. Nhằm tối ưu hóa thiết kế và tiết kiệm tài nguyên chân cắm GPIO của vi điều khiển ESP32-S3, dự án sử dụng loại màn hình LCD đã được tích hợp sẵn chuẩn giao tiếp I2C (Inter-Integrated Circuit). Việc sử dụng màn hình LCD I2C cho phép thiết bị kết nối trực tiếp vào cổng Grove I2C có sẵn trên bo mạch Yolo UNO thông qua một cáp kết nối (Grove cable) duy nhất. Giải pháp này rút gọn toàn bộ giao tiếp điều khiển phức tạp ban đầu xuống chỉ còn bốn dây tín hiệu cơ bản: VCC, GND, SDA (Dữ liệu nối tiếp) và SCL (Xung nhịp nối tiếp). Trong kiến trúc phần mềm, màn hình LCD được FreeRTOS cấp quyền truy cập an toàn thông qua cơ chế Mutex/Semaphore để hiển thị rõ ràng ít nhất 3 trạng thái cảnh báo của hệ thống.



Hình 3: Màn hình hiển thị LCD I2C cắm trực tiếp vào Yolo UNO qua cáp Grove

Bảng 6: Thông số kỹ thuật của màn hình hiển thị LCD I2C

Thông số	Giá trị
Điện áp hoạt động cấp vào	5VDC
Giao thức truyền dữ liệu chính	I2C (Chế độ Master-Slave)
Kết nối vật lý	Cáp cắm Grove 4 chân (VCC, GND, SDA, SCL)
Địa chỉ I2C mặc định	0x27 hoặc 0x3F
Đèn nền hiển thị (Backlight)	Đèn LED tích hợp điều khiển qua phần mềm

Bảng 7: Chức năng các chân giao tiếp của màn hình LCD I2C

STT	Chân	Chức năng
1	GND	Nối đất
2	VCC	Cấp nguồn dương 5VDC
3	SDA	Dây truyền nhận dữ liệu nối tiếp (Serial Data)
4	SCL	Dây cấp xung nhịp nối tiếp (Serial Clock)

2.1.6 Giao thức truyền thông và kết nối

Hệ thống IoT đòi hỏi sự linh hoạt trong cả truyền thông cục bộ cận biên lẫn truyền thông diện rộng lên máy chủ Internet:

- **Giao thức I2C (Inter-Integrated Circuit):** Giao thức truyền thông nối tiếp, đồng bộ, hoạt động ở chế độ Master-Slave trên hai dây bus: SDA (Serial Data Line) và SCL (Serial Clock Line). I2C hỗ trợ đa cấu hình thiết bị trên cùng một bus nhờ cơ chế định địa chỉ vật lý (Address-based signaling), lý tưởng cho việc điều khiển màn hình LCD cục bộ và giao tiếp với cảm biến DHT20.
- **Chuẩn kết nối Wi-Fi (IEEE 802.11):** Hoạt động trên dải tần 2.4GHz, đóng vai trò hạ tầng kết nối không dây chủ đạo của hệ thống nhúng. ESP32 S3 hỗ trợ cấu hình Wi-Fi linh hoạt bao gồm chế độ Điểm truy cập (Access Point - AP) để thiết lập mạng cục bộ riêng biệt cho máy chủ Web, và chế độ Trạm (Station - STA) để lấy IP từ bộ định tuyến và thiết lập liên kết TCP/IP ra Internet.
- **Giao thức mạng HTTP, WebSocket và MQTT:** Phục vụ lớp ứng dụng trong kiến trúc IoT. Giao thức HTTP/WebSocket hoạt động theo mô hình truyền tải trực tiếp thích hợp cho Web Server và giao diện điều khiển (Dashboard) tại chỗ. Giao thức MQTT (Message Queuing Telemetry Transport) hoạt động theo mô hình Publish-Subscribe gọn nhẹ, giảm thiểu băng thông đường truyền, tối ưu cho các tác vụ đẩy dữ liệu liên tục từ cảm biến lên đám mây Cloud.

2.2 FreeRTOS

2.2.1 Quản lý tác vụ (Task Management)

FreeRTOS là một hệ điều hành thời gian thực mã nguồn mở được thiết kế tối ưu cho vi điều khiển. Trong FreeRTOS, một ứng dụng được cấu thành từ nhiều tác vụ (Tasks) độc lập. Mỗi Task là một luồng thực thi (Thread) riêng biệt sở hữu một không gian stack và mức độ ưu tiên (Priority) được cấu hình độc lập tại thời điểm khởi tạo thông qua hàm `xTaskCreate()`. FreeRTOS quản lý vòng đời của một tác vụ qua 4 trạng thái cốt lõi: Running (Đang thực thi), Ready (Sẵn sàng), Blocked (Bị chặn/Chờ sự kiện), và Suspended (Bị tạm dừng).

2.2.2 Cơ chế lập lịch thời gian thực

Bộ lập lịch (Scheduler) của FreeRTOS đóng vai trò quyết định tác vụ nào ở trạng thái Ready sẽ được chuyển sang trạng thái Running. Hệ thống sử dụng thuật toán lập lịch ưu tiên trung dụng dựa trên lát cắt thời gian (Preemptive Priority-Based Scheduling with Time Slicing). Hệ điều hành duy trì một bộ đếm xung nhịp hệ thống gọi là System Tick. Cứ sau mỗi chu kỳ Tick, Bộ lập lịch sẽ quét danh sách các tác vụ Ready; tác vụ nào có mức độ ưu tiên cao nhất sẽ giành quyền kiểm soát CPU (Context Switch). Nếu các tác vụ có mức độ ưu tiên bằng nhau, bộ lập lịch sẽ luân chuyển quyền thực thi theo cơ chế vòng xoay (Round-Robin).

2.2.3 Giao tiếp và đồng bộ giữa các task

Khi nhiều tác vụ chạy đồng thời và chia sẻ tài nguyên phần cứng (như bus I2C của LCD hoặc biến toàn cục), hiện tượng tranh chấp tài nguyên (Race Condition) rất dễ xảy ra nếu không có cơ chế đồng bộ hóa an toàn:

- **Semaphore / Mutex:** Hoạt động như một chiếc thẻ bài (Token). Tác vụ muốn truy cập tài nguyên phải thực hiện hàm lấy thẻ (`xSemaphoreTake()`); nếu tài nguyên đang bận, tác vụ sẽ rơi vào trạng thái Blocked để nhường CPU cho tác vụ khác. Sau khi xử lý xong tài nguyên, tác vụ thực hiện giải phóng thẻ (`xSemaphoreGive()`). Dự án ứng dụng Semaphore và Mutex để đồng bộ hóa tác vụ cảnh báo LED, NeoPixel theo chu kỳ nhiệt độ/độ ẩm và điều phối quyền ghi dữ liệu lên LCD mà không dùng biến toàn cục.
- **Tín hiệu (Signals/Task Notifications):** Cơ chế truyền phát sự kiện trực tiếp đến một tác vụ cụ thể mà không cần qua các đối tượng trung gian. Việc sử dụng Task Notifications giải phóng đáng kể tài nguyên RAM và tối ưu hóa tốc độ chuyển đổi ngữ cảnh.

2.2.4 Quản lý bộ nhớ

FreeRTOS phân tách việc quản lý bộ nhớ thành hai vùng chính: bộ nhớ tĩnh (Static memory) và bộ nhớ động (Heap memory). FreeRTOS cung cấp nhiều mô hình quản

lý bộ nhớ từ `heap_1` đến `heap_5`. Trong dự án phát triển IoT kết hợp TinyML, mô hình `heap_4` thường được lựa chọn nhờ thuật toán gộp các khối bộ nhớ trống liền kề để giảm thiểu hiện tượng phân mảnh bộ nhớ (Memory Fragmentation), đảm bảo vùng nhớ cấp phát động cho các cấu trúc dữ liệu mạng luôn ổn định.

2.2.5 Software Timer

Software Timer (Bộ định thời phần mềm) cho phép thực thi một hàm chức năng (Callback function) sau một khoảng thời gian xác định, hoạt động dựa trên xung nhịp System Tick của hệ điều hành mà không cần sử dụng bộ định thời phần cứng (Hardware Timer) của chip. Timer có thể cấu hình ở chế độ chạy một lần (One-shot) hoặc lặp lại chu kỳ (Auto-reload), rất hữu ích cho các tác vụ định kỳ như quét trạng thái mạng Wi-Fi hoặc cập nhật thời gian hệ thống.

2.2.6 Tiết kiệm năng lượng

FreeRTOS hỗ trợ tính năng `configUSE_IDLE_HOOK` và chế độ Tickless Idle. Khi hệ thống nhận diện tất cả các tác vụ người dùng đều đang ở trạng thái Blocked và không có tác vụ nào sẵn sàng thực thi, bộ lập lịch sẽ chuyển sang thực thi tác vụ rỗi (Idle Task). Tại đây, vi điều khiển có thể được cấu hình để đưa vào các chế độ tiết kiệm năng lượng sâu (Light Sleep hoặc Deep Sleep), ngắt các xung nhịp ngoại vi không cần thiết và chỉ thức dậy khi có ngắt phần cứng hoặc khi bộ đếm thời gian chạm ngưỡng quy định.

2.2.7 Khả năng mở rộng và tương thích

Kiến trúc của FreeRTOS được thiết kế theo dạng mô-đun hóa cao, lớp trừu tượng phần cứng (Hardware Abstraction Layer - HAL) độc lập giúp hệ điều hành dễ dàng tương thích và mở rộng trên nhiều dòng cấu trúc vi điều khiển khác nhau (từ lõi ARM Cortex-M cho đến kiến trúc Xtensa Tensilica dual-core của ESP32 S3). Điều này cho phép nhà phát triển dễ dàng tích hợp thêm các thư viện phần mềm bên thứ ba mà không phá vỡ cấu trúc lập lịch thời gian thực hiện có.

2.2.8 Các thư viện hỗ trợ IoT

Để hỗ trợ toàn diện cho các ứng dụng IoT phức tạp, FreeRTOS cung cấp hệ sinh thái thư viện phong phú bao gồm: thư viện lõi FreeRTOS-Plus-TCP hỗ trợ tầng mạng, các thư viện bảo mật lớp truyền tải như mbedTLS phục vụ mã hóa dữ liệu đầu cuối, và các thư viện giao tiếp mạng được tối ưu hóa bộ nhớ stack, giúp thiết bị nhúng giao tiếp mượt mà với các Broker cục bộ hoặc máy chủ đám mây.

2.3 CoreIOT

2.3.1 Các chức năng chính của CoreIOT

CoreIOT là một nền tảng Đám mây ứng dụng công nghiệp (Industrial IoT Cloud Platform) chuyên dụng, cung cấp giải pháp đầu cuối cho việc quản lý, giám sát và phân tích dữ liệu thiết bị thông minh. Các chức năng cốt lõi bao gồm:

- Tiếp nhận và lưu trữ có cấu trúc các chuỗi dữ liệu thời gian thực gửi về từ thiết bị nhúng.
- Hỗ trợ cơ chế định danh, quản lý vòng đời và xác thực bảo mật thiết bị kết nối.
- Tích hợp hệ thống phân tích dữ liệu, tự động kích hoạt các kịch bản cảnh báo (E-mail, SMS, Webhook) khi thông số vượt ngưỡng.

2.3.2 Kiến trúc hệ thống

Kiến trúc của nền tảng CoreIOT được xây dựng dựa trên mô hình ba tầng chuẩn mô hình IoT đám mây:

1. **Tầng kết nối giao tiếp (Ingestion Layer):** Đóng vai trò là cổng tiếp nhận (Gateway API), hỗ trợ các giao thức mạng phổ biến như RESTful HTTP, MQTT bảo mật.
2. **Tầng xử lý và lưu trữ dữ liệu (Processing & Storage Layer):** Sử dụng các kiến trúc cơ sở dữ liệu chuỗi thời gian (Time-Series Database) hiệu năng cao để tối ưu tốc độ ghi dữ liệu liên tục từ hàng vạn cảm biến.

3. **Tầng ứng dụng và hiển thị (Application/Presentation Layer):** Cung cấp các giao diện lập trình ứng dụng (API REST) kết hợp hệ thống giao diện Dashboard tương tác trực quan cho người dùng cuối.

2.3.3 Đặc điểm nổi bật

CoreIOT nổi bật với khả năng tối ưu hóa băng thông đường truyền cực tốt nhờ việc hỗ trợ các khuôn mẫu gói tin JSON thu gọn, giao diện lập trình thiết kế kéo thả Dashboard linh hoạt, trực quan mà không đòi hỏi hạ tầng máy chủ phức tạp. Đặc biệt, hệ thống cung cấp các giải pháp mẫu (Solution Templates) được định cấu hình sẵn, giúp các kỹ sư nhúng nhanh chóng kết nối phần cứng thông qua mã xác thực thiết bị duy nhất (Device Token), tăng tốc độ triển khai dự án thực tế.

2.4 TinyML

2.4.1 Đặc điểm của TinyML

TinyML (Tiny Machine Learning) là một lĩnh vực nghiên cứu kết hợp giữa hệ thống nhúng và học máy, hướng tới việc thực thi các mô hình mạng học sâu và thuật toán học máy trực tiếp trên các thiết bị phần cứng có công suất tiêu thụ cực thấp (dưới mức miliwatt) và tài nguyên bộ nhớ vô cùng hạn chế (chỉ vài trăm Kilobytes RAM). Khác với các mô hình AI truyền thống đòi hỏi trung tâm dữ liệu đám mây mạnh mẽ, TinyML đưa năng lực tính toán thông minh về sát cạnh biên, loại bỏ hoàn toàn độ trễ đường truyền mạng, bảo mật thông tin tuyệt đối và cho phép thiết bị hoạt động liên tục ngay cả khi mất kết nối Internet hoàn toàn.

2.4.2 Kiến trúc hoạt động của TinyML

Quy trình thực thi một kiến trúc TinyML trên thiết bị biên tuân thủ chặt chẽ các giai đoạn sau:

1. **Thu thập dữ liệu tại biên (Data Ingestion):** Dữ liệu thô liên tục được trích xuất từ cảm biến với tần số lấy mẫu cố định.
2. **Tiền xử lý và trích xuất đặc trưng (Feature Extraction):** Loại bỏ nhiễu tín hiệu. Tùy thuộc vào thiết kế mô hình, dữ liệu có thể được chuẩn hóa về khoảng

$[0, 1]$ hoặc $[-1, 1]$, hoặc được nạp thẳng dưới dạng dữ liệu số thực gốc để tối ưu chu trình tính toán.

3. Suy luận mô hình (Inference): Dữ liệu đặc trưng được đưa qua các lớp mạng nơ-ron (như CNN hoặc Dense Layers) để tính toán ma trận và đưa ra kết quả phân loại trạng thái. Để tăng tốc độ, mô hình có thể được lượng tử hóa (Quantization) hoặc giữ nguyên cấu trúc số thực (float32).

2.4.3 TensorFlow Lite Micro

TensorFlow Lite Micro là khung sườn phần mềm (Framework) học máy mã nguồn mở được thiết kế riêng bởi Google để chạy các mô hình học sâu trên các chip vi điều khiển kiến trúc 32-bit. Khung sườn này được tối ưu hóa triệt để để không sử dụng bất kỳ cơ chế cấp phát bộ nhớ động nào của hệ điều hành (không dùng `malloc` hay các toán tử `new`) nhằm tránh tuyệt đối lỗi phân mảnh RAM và đảm bảo tính thực thi an toàn cao. Toàn bộ các mảng trọng số, lớp mạng và biến trung gian của mô hình đều được quản lý tập trung trong một mảng bộ nhớ tĩnh được cấp phát từ trước gọi là **Tensor Arena**. Các toán tử tính toán trong thư viện được tối ưu hóa toán học để tận dụng tối đa tập lệnh phần cứng của chip xử lý, giúp tăng tốc độ nhân ma trận.

2.4.4 Ứng dụng của TinyML trong IoT

Trong các hệ thống IoT thông minh, TinyML đóng vai trò là "bộ não" cục bộ giúp nâng cao giá trị dữ liệu thô. Thay vì liên tục gửi dữ liệu cảm biến chưa qua xử lý lên đám mây gây lãng phí băng thông mạng và năng lượng pin, TinyML thực hiện phân tích, nhận diện mẫu hành vi, dự đoán xu hướng hư hỏng của máy móc, hoặc phát hiện các bất thường về nhiệt độ, môi trường ngay tại chỗ. Thiết bị chỉ phát tín hiệu mạng hoặc đẩy dữ liệu lên Cloud khi mô hình TinyML phát hiện ra các sự kiện thực sự quan trọng hoặc các tình huống khẩn cấp, tối ưu hóa toàn diện hiệu suất vận hành của toàn mạng lưới IoT.

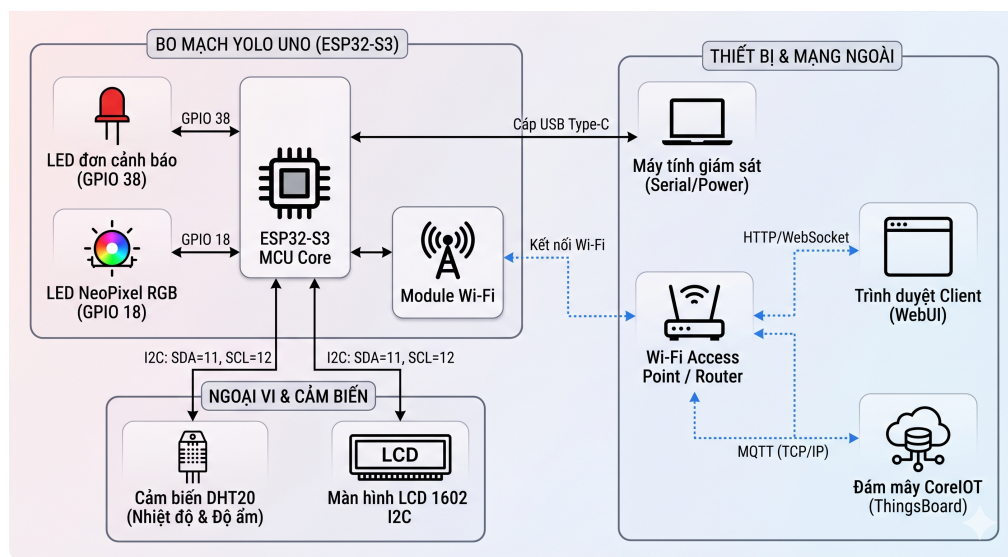
3 Thiết kế hệ thống

3.1 Tổng quan hệ thống

Để đáp ứng yêu cầu xử lý thời gian thực, đồng bộ hóa luồng dữ liệu phức tạp từ cảm biến và tối ưu hóa tài nguyên phần cứng, hệ thống được thiết kế dựa trên kiến trúc **đa tác vụ (Multi-tasking)** chạy trên nền tảng **Hệ điều hành thời gian thực FreeRTOS** nhúng trong vi điều khiển lõi kép **ESP32-S3 (Yolo UNO)**.

Toàn bộ các tác vụ xử lý bao gồm đọc dữ liệu cảm biến, hiển thị LCD, điều khiển LED/NeoPixel cảnh báo, thực thi mô hình trí tuệ nhân tạo biên (TinyML), chạy máy chủ Web Server cục bộ và đồng bộ dữ liệu đám mây (CoreIOT) được thiết kế chạy song song độc lập. Các tác vụ này tương tác và chia sẻ tài nguyên một cách an toàn thông qua các cơ chế đồng bộ hóa luồng của FreeRTOS như **Mutex** và **Semaphore**, giúp loại bỏ hoàn toàn các biến toàn cục không an toàn.

Sơ đồ kết nối phần cứng chi tiết của hệ thống được thể hiện trong Hình 4.

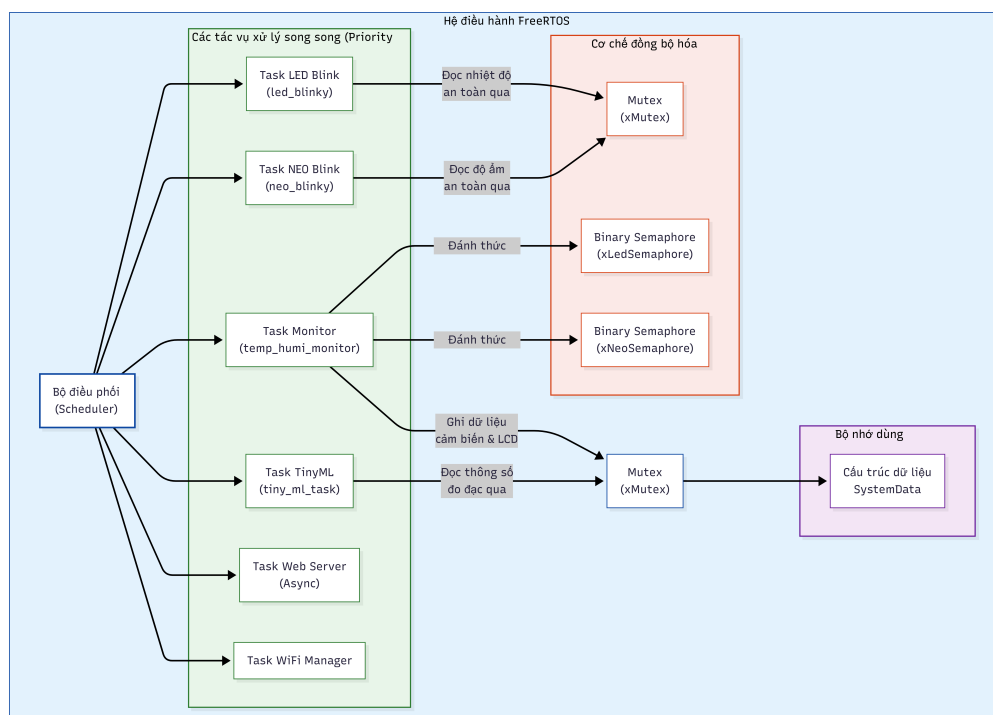


Hình 4: Sơ đồ khối tổng quan kết nối phần cứng hệ thống

Qua sơ đồ này, vi điều khiển ESP32-S3 đóng vai trò là bộ xử lý trung tâm, thu thập dữ liệu từ cảm biến DHT20 qua giao thức I2C (sử dụng hai đường truyền tín hiệu SDA tại chân GPIO 11 và SCL tại chân GPIO 12) và chỉ thị trạng thái ra màn hình LCD 1602 (cũng kết nối chung bus I2C). Các cơ cấu hiển thị cảnh báo cục bộ khác

gồm đèn LED đơn cảnh báo được điều khiển qua chân GPIO 38 và LED NeoPixel RGB qua chân GPIO 18. Thiết bị được cấp nguồn và giao tiếp gỡ lỗi với máy tính giám sát thông qua cáp USB Type-C. Đồng thời, module Wi-Fi tích hợp thiết lập liên kết không dây với Router trong mạng nội bộ để hỗ trợ giao tiếp WebSocket với các thiết bị khách (Web Client) và giao thức MQTT với máy chủ đám mây CoreIOT (ThingsBoard).

Kiến trúc đa tác vụ song song của hệ thống dưới sự quản lý của FreeRTOS được mô tả chi tiết trong Hình 5.



Hình 5: Kiến trúc đa tác vụ hoạt động song song dưới sự điều phối của FreeRTOS trên bo mạch Yolo UNO

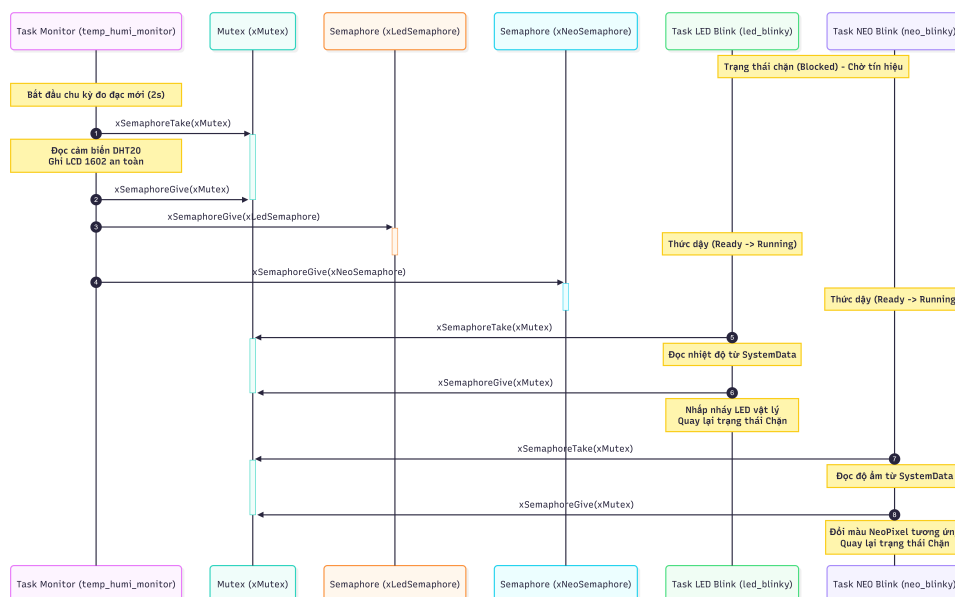
Kiến trúc này thể hiện việc hệ thống phân chia công việc thành 6 tác vụ (Tasks) chạy song song với độ ưu tiên bằng nhau (Priority 2): Task LED Blink (chớp tắt LED đơn), Task NEO Blink (điều khiển màu dải NeoPixel), Task Monitor (đọc cảm biến DHT20 và cập nhật LCD), Task TinyML (chạy mô hình suy luận biên), Task Web Server (lắng nghe yêu cầu HTTP/WebSocket không đồng bộ), và Task WiFi Manager (quản lý kết nối mạng). Các tác vụ này tương tác gián tiếp với nhau thông qua cấu trúc dữ

liệu dùng chung `SystemData`. Để loại bỏ hoàn toàn hiện tượng tranh chấp tài nguyên (Race Condition) trên bus I2C và vùng nhớ dùng chung, hệ thống áp dụng cơ chế đồng bộ hóa thông qua Mutex (`xMutex`) bảo vệ. Ngoài ra, Task Monitor đóng vai trò là nguồn phát tín hiệu, định kỳ giải phóng các Semaphore nhị phân (`xLedSemaphore` và `xNeoSemaphore`) để đánh thức các tác vụ cảnh báo LED đơn và NeoPixel khi có dữ liệu cảm biến mới.

3.2 Khối hiển thị LED, NeoPixel và LCD (Cảnh báo cục bộ)

Khối chức năng hiển thị và cảnh báo cục bộ chịu trách nhiệm tương tác trực quan với người dùng tại chỗ. Để tránh xung đột tài nguyên và đồng bộ hóa các chu kỳ hiển thị theo chu kỳ đọc cảm biến, hệ thống thiết kế các tác vụ riêng biệt được đồng bộ hóa thời gian thực:

- **Tác vụ điều khiển LED đơn (Task 1):** Tác vụ này duy trì trạng thái chặn (Blocked State) để tiết kiệm CPU và chỉ thức dậy khi nhận được tín hiệu giải phóng từ `**Binary Semaphore**` (`xLedSemaphore`) từ tác vụ đọc cảm biến. Tác vụ sẽ kiểm tra giá trị nhiệt độ hiện tại để điều khiển tần số nhấp nháy của LED theo 3 dải cảnh báo riêng biệt (an toàn, cảnh báo, khẩn cấp).
- **Tác vụ điều khiển LED NeoPixel (Task 2):** Tương tự như LED đơn, tác vụ điều khiển màu sắc của dải LED RGB NeoPixel được đồng bộ qua `**Binary Semaphore**` (`xNeoSemaphore`). Tác vụ ánh xạ mức độ ẩm nhận được thành 3 màu sắc đại diện (Đỏ - khô hạn, Xanh lá - lý tưởng, Xanh dương - ẩm ướt) mà không gây trễ giao diện.
- **Tác vụ hiển thị LCD I2C (Task 3):** Tác vụ này kiểm soát màn hình LCD 1602 giao tiếp qua I2C. Để ngăn ngừa hiện tượng tranh chấp đường truyền bus I2C (Race Condition) khi ghi thông số cảm biến, hệ thống sử dụng `**Mutex**` (`xMutex`) để độc chiếm tài nguyên bus I2C một cách an toàn. Màn hình tự động cập nhật nhiệt độ, độ ẩm thực tế và thay đổi linh hoạt giữa 3 trạng thái giao diện: “NORMAL”, “WARNING”, và “CRITICAL”.



Hình 6: Cơ chế đồng bộ hóa đa tác vụ cảnh báo cục bộ qua Semaphore và Mutex

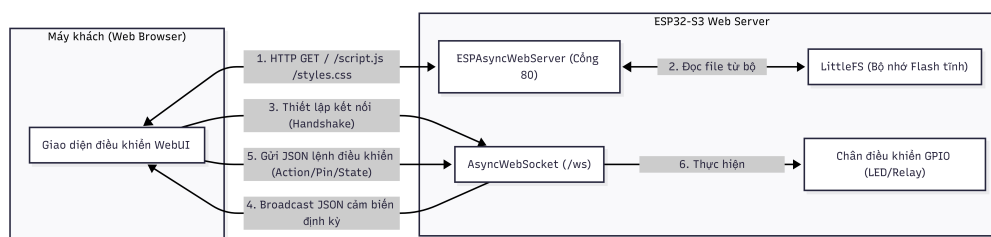
Cơ chế đồng bộ hóa chi tiết giữa ba tác vụ hiển thị và cảnh báo cục bộ thông qua các Semaphore và Mutex được thể hiện trực quan trong Hình 6. Quy trình đồng bộ hóa bắt đầu khi Task Monitor khởi chạy chu kỳ đo đạc mới (mỗi 2 giây). Ban đầu, các Task LED Blink và Task NEO Blink ở trạng thái Chặn (Blocked) để nhường CPU. Task Monitor yêu cầu chiếm Mutex **xMutex** để đọc quyền ghi dữ liệu từ cảm biến DHT20 lên màn hình LCD 1602 một cách an toàn, tránh xung đột bus I2C. Sau khi hoàn tất và trả lại Mutex, Task Monitor thực hiện gọi **xSemaphoreGive** cho hai Semaphore nhị phân **xLedSemaphore** và **xNeoSemaphore** để kích hoạt (đánh thức) các tác vụ phụ. Khi thức dậy, Task LED Blink and Task NEO Blink lần lượt chiếm Mutex **xMutex** trong khoảng thời gian rất ngắn để đọc giá trị nhiệt độ, độ ẩm an toàn từ cấu trúc dữ liệu chung, sau đó thực thi điều khiển nhấp nháy LED vật lý hoặc chuyển đổi màu sắc NeoPixel trước khi quay trở lại trạng thái chặn. Thiết kế này đảm bảo việc truy cập tài nguyên LCD không bị xung đột và tần số nhấp nháy/màu sắc của LED được cập nhật ngay lập tức khi có dữ liệu cảm biến mới.

3.3 Khối Webserver + Điều khiển cục bộ (Task 4)

Khối truyền thông điều khiển cục bộ cung cấp một máy chủ Web Server tích hợp trực tiếp trên chip ESP32-S3, cho phép người dùng giám sát và cấu hình thiết bị từ

xa qua mạng nội bộ:

- **Máy chủ Web Asynchronous:** Sử dụng thư viện máy chủ không đồng bộ ('ESPAsyncWebServer'), giúp phản hồi nhanh các yêu cầu HTTP mà không gây nghẽn các tác vụ RTOS khác. Toàn bộ các tệp tin giao diện WebUI được lưu trữ trực tiếp trong bộ nhớ Flash của chip dưới dạng hệ thống tệp tin tĩnh.
- **Giao tiếp WebSocket:** Thay vì sử dụng HTTP REST API truyền thống, hệ thống thiết kế kênh truyền thông song công **WebSocket** ('AsyncWebSocket') để truyền dữ liệu cảm biến dạng JSON theo thời gian thực tới giao diện người dùng và tiếp nhận tức thời lệnh điều khiển cơ cấu chấp hành (bật/tắt các cổng GPIO liên kết với LED/Relay).



Hình 7: Sơ đồ giao tiếp truyền thông cục bộ qua WebServer và WebSocket

Quy trình truyền nhận dữ liệu thời gian thực và lệnh điều khiển thiết bị thông qua giao thức WebSocket kết hợp máy chủ Web không đồng bộ được minh họa trong Hình 7. Ở giai đoạn khởi động, khi Web Client (Trình duyệt) truy cập địa chỉ thiết bị, máy chủ Web Asynchronous (ESPAsyncWebServer) sẽ tiếp nhận yêu cầu HTTP GET và đọc các tệp tĩnh gồm `index.html`, `script.js`, `styles.css` từ bộ nhớ Flash tĩnh định dạng hệ thống tệp tin LittleFS để trả về cho trình duyệt. Sau khi tải xong trang web, trình duyệt sẽ chủ động khởi tạo một kết nối WebSocket bắt tay (Handshake) thông qua endpoint `/ws` để thiết lập kênh truyền song công liên tục. Thông qua kênh này, Server định kỳ đóng gói dữ liệu cảm biến thành chuỗi JSON và gửi quảng bá (broadcast) tới mọi client đang kết nối để cập nhật đồ thị giao diện thời gian thực. Ngược lại, khi người dùng tương tác trên WebUI, Web Client sẽ gửi gói tin JSON chứa các lệnh điều khiển (`action`, `pin`, `state`) lên server để thực hiện hàm `digitalWrite`

tác động trực tiếp vào các chân GPIO ngoại vi (LED, Relay). WebSocket giúp duy trì kết nối song công liên tục, giảm thiểu băng thông và tăng tốc độ phản hồi lệnh điều khiển từ người dùng.

3.4 Khối CoreIOT + Kết nối mạng STA (Task 6)

Khối tương tác đám mây diện rộng đảm nhận việc đẩy dữ liệu lên máy chủ từ xa phục vụ giám sát tập trung:

- **Cơ chế kết nối WiFi STA:** Khi có cấu hình mạng hợp lệ, thiết bị kích hoạt Wi-Fi ở chế độ Trạm (Station - STA) để liên kết vào mạng bộ định tuyến có kết nối Internet.
- **Đẩy dữ liệu Cloud qua MQTT:** Tác vụ định kỳ đóng gói giá trị nhiệt độ và độ ẩm vào gói tin JSON, sau đó sử dụng thư viện ThingsBoard MQTT kết hợp với Device Token để thực hiện xuất bản (publish) dữ liệu viễn trắc lên máy chủ đám mây CoreIOT (<https://app.coreiot.io/>) một cách an toàn.

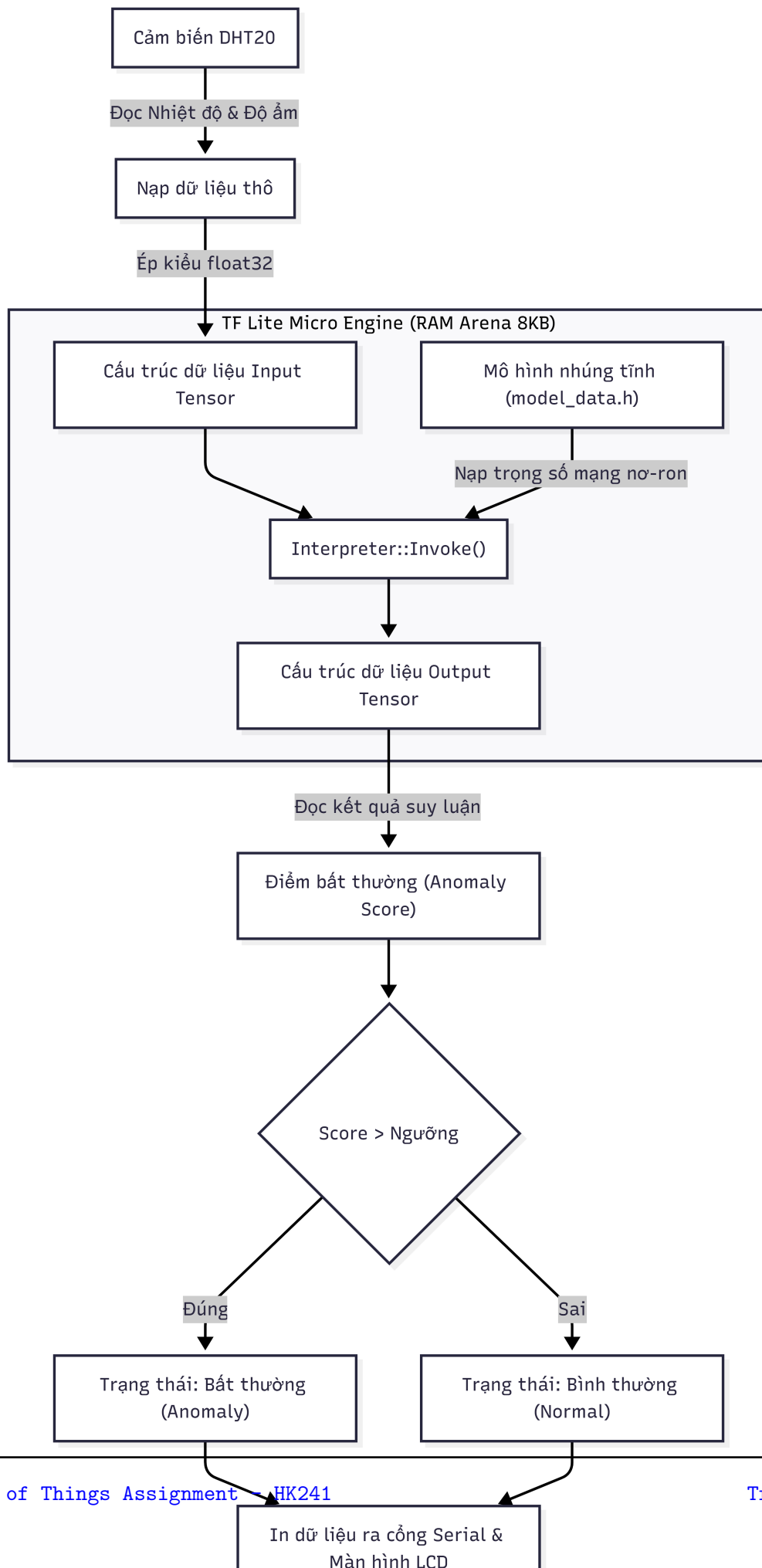
3.5 Khối TinyML (Task 5)

Khối trí tuệ nhân tạo nhúng chạy trực tiếp thuật toán học máy tại biên để phát hiện bất thường môi trường:

- **Mô hình TensorFlow Lite Micro:** Mô hình mạng nơ-ron lan truyền thẳng (FFN) được huấn luyện từ trước và nhúng trực tiếp dưới dạng mảng hằng số C. Dữ liệu nhiệt độ và độ ẩm thực tế dạng số thực ('float32') được nạp trực tiếp làm đầu vào cho mô hình suy luận.
- **Bộ nhớ Tensor Arena tĩnh:** Để tránh tuyệt đối lỗi sập nguồn do phân mảnh bộ nhớ động (Heap fragmentation) trên hệ thống thời gian thực, bộ thông dịch TensorFlow Lite Micro được cấp phát một vùng nhớ tĩnh ****8KB**** cố định trên RAM.
- **Tự đánh giá khi khởi động (Startup Test):** Nhằm đo lường độ chính xác của mô hình ngay trên phần cứng, hệ thống tích hợp tập dữ liệu kiểm thử gồm 50



mẫu tĩnh. Khi khởi động, tác vụ TinyML sẽ chạy suy luận trên toàn bộ tập mẫu này và tính toán các chỉ số chất lượng gồm Accuracy, Precision, Recall và xuất ra cổng Serial.

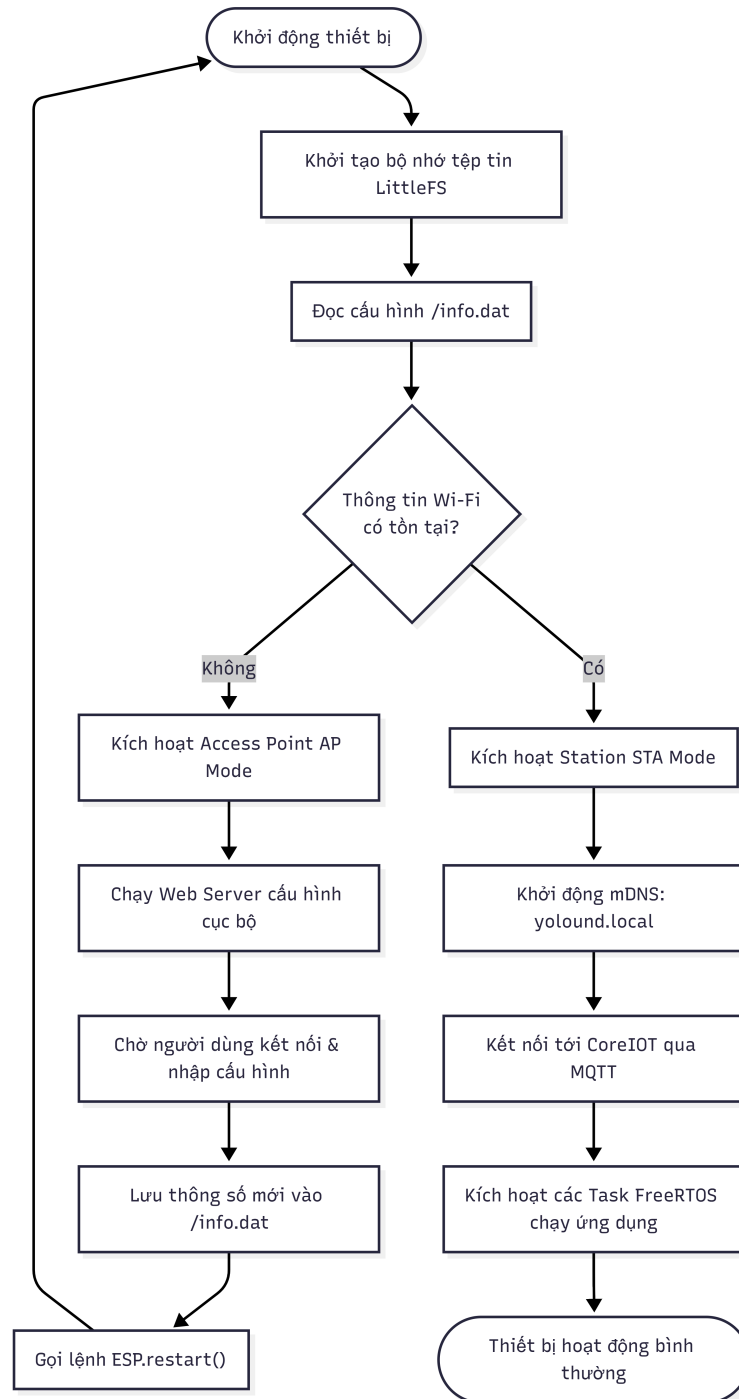


Quy trình suy luận biên của mô hình TinyML được mô tả trong Hình 8. Dữ liệu thô thu thập từ cảm biến DHT20 sẽ được ép kiểu dữ liệu về dạng số thực `float32` trước khi nạp vào vùng nhớ Tensor đầu vào (Input Tensor). Bộ thông dịch TensorFlow Lite Micro đóng vai trò trung tâm xử lý, sử dụng mô hình mạng nơ-ron đã được nhúng tính dưới dạng mảng byte (`model_data.h`) và vùng nhớ đệm Arena tĩnh có kích thước cố định **8KB** trên RAM để tiến hành thực thi hàm `interpreter->Invoke()`. Sau khi tính toán lan truyền thẳng qua các lớp mạng nơ-ron, kết quả suy luận được lưu vào Tensor đầu ra (Output Tensor) dưới dạng một Điểm bất thường (Anomaly Score). Hệ thống sẽ so sánh điểm số này với một ngưỡng cài đặt trước (0.5): nếu điểm số vượt ngưỡng, thiết bị xác định trạng thái môi trường là Bất thường (Anomaly), ngược lại là Bình thường (Normal). Kết quả này sẽ được xuất trực tiếp ra cổng Serial và cập nhật lên màn hình LCD cục bộ.

3.6 Khối quản lý cấu hình và Định danh thiết bị (LittleFS)

Hệ thống được thiết kế để hoạt động động và linh hoạt mà không cần nạp lại chương trình khi thay đổi môi trường:

- **Lưu trữ cấu hình động bằng LittleFS:** Toàn bộ thông tin kết nối Wi-Fi (SSID, Password) và thông tin cấu hình đám mây CoreIOT (Server, Token, Port) được lưu trữ dưới dạng tệp tin JSON `"/info.dat"` trong phân vùng LittleFS trên bộ nhớ Flash. Khi khởi động, hệ thống đọc tệp tin này; nếu chưa cấu hình, thiết bị sẽ tự động khởi tạo Access Point phát Wi-Fi để người dùng nhập thông tin. Sau khi nhận được thông tin cấu hình từ WebUI, thiết bị lưu lại và tự khởi động lại vào chế độ STA kết nối Internet.
- **Định danh bằng MAC Address:** Hệ thống sử dụng địa chỉ vật lý MAC Address mặc định của phần cứng chip Wi-Fi để tạo thuộc tính định danh thiết bị độc bản gửi lên CoreIOT, giúp quản lý và định danh chính xác thiết bị trên máy chủ.



Hình 9: Lưu đồ thuật toán quản lý cấu hình động bằng LittleFS

Lưu đồ thuật toán quản lý cấu hình và khởi động động của hệ thống sử dụng hệ thống tệp tin LittleFS được mô tả trong Hình 9. Ngay khi thiết bị được khởi động, hệ thống tiến hành khởi tạo phân vùng tệp tin LittleFS và tìm đọc tệp dữ liệu cấu hình cục bộ /info.dat. Tiếp theo, một khối điều kiện kiểm tra sự tồn tại của thông tin kết nối Wi-Fi:

- Nếu thông tin chưa tồn tại (như khi vận hành lần đầu tiên), thiết bị sẽ tự động kích hoạt chế độ Access Point (AP Mode), phát mạng Wi-Fi cấu hình và khởi chạy máy chủ Web cục bộ. Thiết bị duy trì trạng thái chờ người dùng kết nối vào mạng để nhập thông số cấu hình mạng mới thông qua trang WebUI. Sau khi nhận được thông tin, hệ thống ghi đè cấu hình vào `/info.dat` và gọi lệnh restart phần cứng để bắt đầu chu kỳ khởi chạy mới.
- Nếu thông tin Wi-Fi đã tồn tại, thiết bị lập tức kích hoạt chế độ Station (STA Mode) kết nối trực tiếp vào mạng nội bộ, khởi tạo dịch vụ tên miền mDNS `yolound.local`, kết nối MQTT tới máy chủ CoreIOT và kích hoạt các tác vụ RTOS để chuyển sang chế độ hoạt động bình thường.

Cơ chế này giúp thiết bị linh hoạt tự động chuyển đổi giữa chế độ cấu hình (AP Mode) và chế độ hoạt động bình thường (STA Mode).

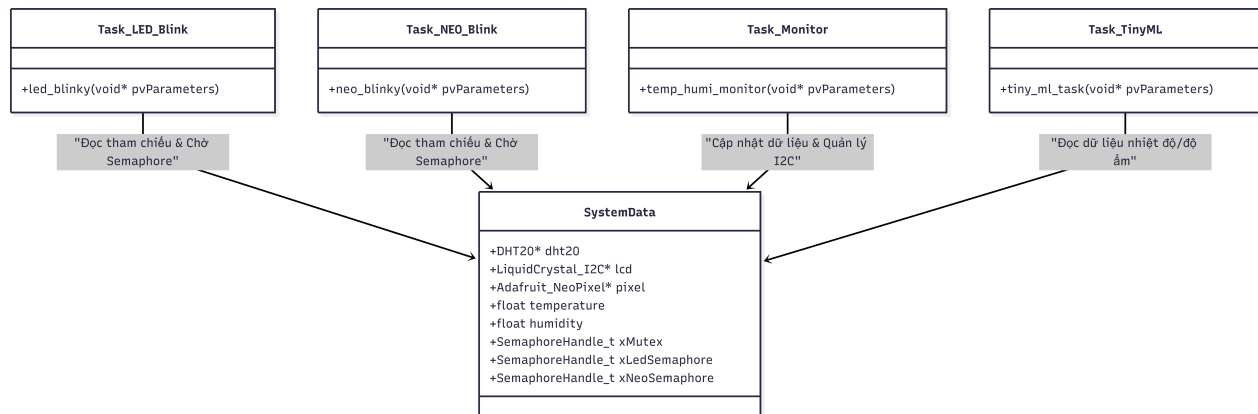
4 Hiện thực hệ thống

4.1 Kiến trúc Đa nhiệm FreeRTOS trên Yolo UNO

Hệ thống được thiết kế chạy trên một bo mạch Yolo UNO duy nhất với chip xử lý kép ESP32-S3. Để phân phối tài nguyên và chạy song song các khối chức năng, toàn bộ mã nguồn được hiện thực dưới dạng các tác vụ (Tasks) của hệ điều hành thời gian thực FreeRTOS. Tại hàm khởi tạo `setup()`, các tài nguyên đồng bộ bao gồm Mutex và các Semaphore nhị phân được khởi tạo trước khi phân phối vào từng tác vụ:

```
1 // Initialize SystemData resources
2 systemData.dht20 = &dht20_inst;
3 systemData.lcd = &lcd_inst;
4 systemData.pixel = &pixel_inst;
5
6 systemData.xMutex = xSemaphoreCreateMutex();
7 systemData.xLedSemaphore = xSemaphoreCreateBinary();
8 systemData.xNeoSemaphore = xSemaphoreCreateBinary();
9 xSerialMutex = xSemaphoreCreateMutex();
10
11 xTaskCreate(led_blinky, "Task LED Blink", 4096, &systemData, 2, NULL);
12 xTaskCreate(neo_blinky, "Task NEO Blink", 4096, &systemData, 2, NULL);
13 xTaskCreate(temp_humi_monitor, "Task TEMP HUMI Monitor", 4096, &systemData, 2, NULL
    );
14 xTaskCreate(tiny_ml_task, "Tiny ML Task", 8192, &systemData, 2, NULL);
```

Để loại bỏ hoàn toàn các biến toàn cục không an toàn, tất cả các tác vụ đều nhận dữ liệu và tài nguyên thông qua con trỏ cấu trúc `SystemData` được truyền vào tham số `pvParameters`.



Hình 10: Cấu trúc sơ đồ dữ liệu SystemData và phân bổ tài nguyên tác vụ

Sơ đồ cấu trúc dữ liệu **SystemData** chia sẻ giữa các tác vụ được thể hiện trong Hình 10. Trong kiến trúc này, lớp cấu trúc dữ liệu trung tâm đóng gói toàn bộ tài nguyên dùng chung, bao gồm các con trỏ tới các đối tượng ngoại vi phần cứng vật lý (cảm biến `DHT20* dht20`, màn hình `LiquidCrystal_I2C* lcd`, dải đèn `Adafruit_NeoPixel* pixel`), các biến lưu trữ thông số đo đạc thời gian thực (`temperature`, `humidity`), và các handle điều phối đồng bộ của FreeRTOS (`xMutex`, `xLedSemaphore`, `xNeoSemaphore`). Các tác vụ chấp hành phụ độc lập như `Task_LED_Blink`, `Task_NEO_Blink`, `Task_Monitor`, và `Task_TinyML` đều được truyền tham số đầu vào là con trỏ dẫn tới cấu trúc `SystemData` này để truy xuất tài nguyên một cách an toàn và tránh sử dụng các biến toàn cục không an toàn.

4.2 Hiện thực Khối cảnh báo và hiển thị ngoại vi

4.2.1 Tác vụ điều khiển LED đơn (Task 1)

Tác vụ quản lý đèn LED đơn chỉ thị cảnh báo nhiệt độ (`led_blinky`) được ghim trong một vòng lặp vô hạn. Tác vụ duy trì trạng thái chặn (*Blocked State*) nhằm tiết kiệm tài nguyên CPU bằng cách chờ đợi tín hiệu từ tác vụ đo đạc thông qua `xLedSemaphore`:

```
1 if (xSemaphoreTake(data->xLedSemaphore, portMAX_DELAY) == pdTRUE) {
2     // Safely access temperature data using Mutex
3     if (xSemaphoreTake(data->xMutex, portMAX_DELAY) == pdTRUE) {
4         float current_temp = data->temperature;
```

```
5     xSemaphoreGive(data->xMutex);
6
7     // Define 3 blinking behaviors based on temperature
8     if (current_temp < 30.0) {
9         blink_delay = 2000;
10    }
11    else if (current_temp >= 30.0 && current_temp < 35.0) {
12        blink_delay = 1000;
13    }
14    else {
15        blink_delay = 200;
16    }
17 }
18 }
```

Sau khi giải phóng Mutex, tác vụ sẽ nhấp nháy đèn LED vật lý (LED_GPIO) với chu kỳ trễ tương ứng bằng hàm `vTaskDelay()`.

4.2.2 Tác vụ điều khiển NeoPixel (Task 2)

Tương tự tác vụ LED đơn, việc đồng bộ dải LED RGB NeoPixel (`neo_blinky`) sử dụng cơ chế Semaphore nhị phân để tối ưu hóa phản hồi chu kỳ:

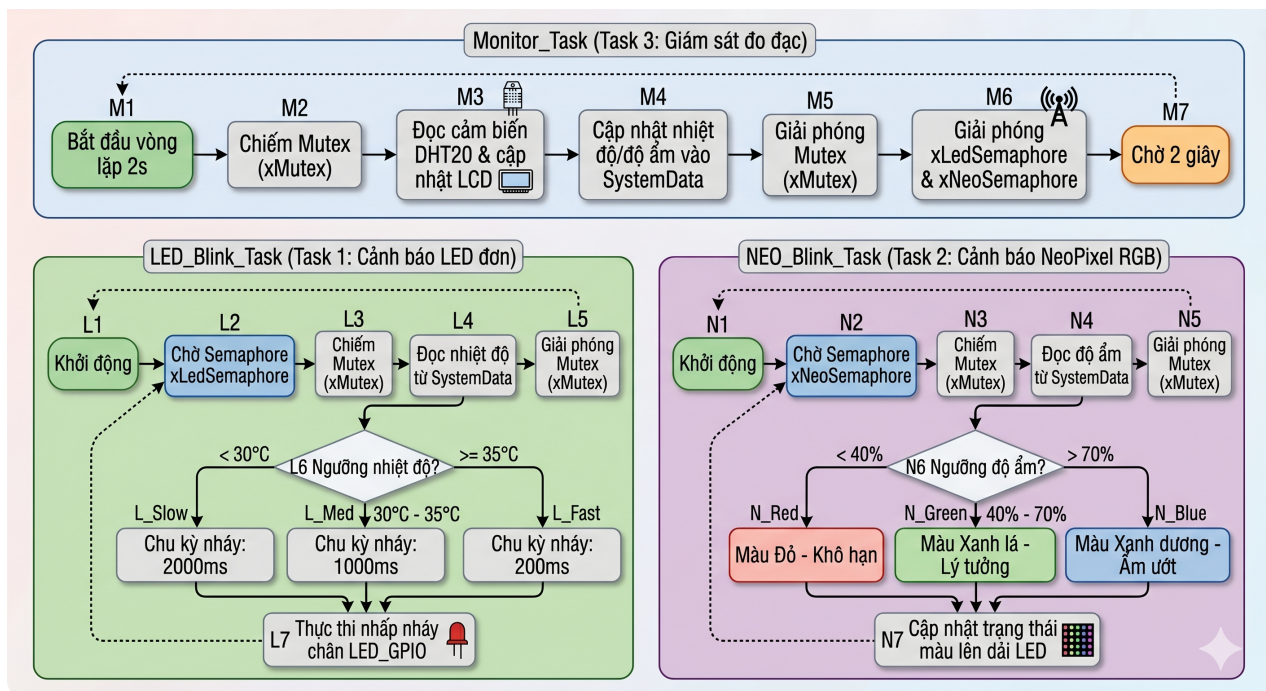
```
1 if (xSemaphoreTake(data->xNeoSemaphore, portMAX_DELAY) == pdTRUE) {
2     float current_humi = 0;
3     // Safely access humidity data using Mutex
4     if (xSemaphoreTake(data->xMutex, portMAX_DELAY) == pdTRUE) {
5         current_humi = data->humidity;
6         xSemaphoreGive(data->xMutex);
7     }
8
9     // Define 3 humidity levels/colors
10    if (current_humi < 40.0) {
11        data->pixel->setPixelColor(0, data->pixel->Color(255, 0, 0)); // Red (Dry)
12    }
13    else if (current_humi >= 40.0 && current_humi <= 70.0) {
14        data->pixel->setPixelColor(0, data->pixel->Color(0, 255, 0)); // Green (
15        Normal)
16    }
17 }
```

```
15     }
16     else {
17         data->pixel->setPixelColor(0, data->pixel->Color(0, 0, 255)); // Blue (
Humid)
18     }
19     data->pixel->show();
20 }
```

4.2.3 Tác vụ đo đặc cảm biến và hiển thị LCD (Task 3)

Tác vụ giám sát nhiệt độ - độ ẩm và màn hình hiển thị (`temp_humi_monitor`) thực hiện đọc dữ liệu từ cảm biến DHT20 thông qua bus I2C (cấu hình chân SDA = 11, SCL = 12). Để tránh tranh chấp tài nguyên bus I2C hoặc tranh chấp ghi màn hình LCD 1602 với các tiến trình khác, tác vụ sử dụng `xMutex` để độc chiếm quyền truy cập an toàn. Sau khi thu thập dữ liệu cảm biến thành công, tác vụ sẽ:

1. Cập nhật dữ liệu vào cấu trúc dùng chung `SystemData` dưới sự bảo vệ của `xMutex`.
2. Phân tích ngưỡng nhiệt độ và cập nhật trạng thái hiển thị LCD theo 3 mức cảnh báo rõ rệt (`State: Normal`, `State: Warning`, hoặc `State: Critical`).
3. Giải phóng `xLedSemaphore` và `xNeoSemaphore` để đánh thức Task 1 và Task 2 thực thi phản hồi chỉ thị.
4. Sử dụng `xSerialMutex` để in log kết quả ra cổng Serial, đồng thời đóng gói dữ liệu cảm biến dạng JSON gửi xuống các client WebSocket và đẩy lên đám mây thông qua giao thức MQTT.



Hình 11: Lưu đồ hoạt động của các tác vụ quản lý ngoại vi chỉ thị cục bộ

Lưu đồ hoạt động chi tiết phối hợp giữa các tác vụ điều khiển ngoại vi chỉ thị được mô tả trong Hình 11. Tiến trình gồm 3 nhánh hoạt động song song độc lập được đồng bộ hóa:

- **Monitor Task (Đo đạc):** Định kỳ mỗi 2 giây, tác vụ chiếm Mutex `xMutex`, tiến hành đọc dữ liệu nhiệt độ và độ ẩm từ cảm biến DHT20, ghi dữ liệu hiển thị lên LCD 1602 và cập nhật vào cấu trúc dùng chung `SystemData`. Sau đó, giải phóng Mutex và phát tín hiệu đánh thức hai Semaphore `xLedSemaphore` và `xNeoSemaphore`.
- **LED Blink Task (Cảnh báo LED đơn):** Chờ Semaphore `xLedSemaphore` được giải phóng. Khi được đánh thức, tác vụ chiếm Mutex `xMutex` để đọc giá trị nhiệt độ hiện tại từ `SystemData`, giải phóng Mutex rồi phân loại nhiệt độ thành 3 ngưỡng cảnh báo để quyết định chu kỳ nhấp nháy LED: nhấp chậm (chu kỳ 2000ms) nếu nhiệt độ dưới 30°C , nhấp vừa (chu kỳ 1000ms) nếu nhiệt độ từ 30°C đến dưới 35°C , và nhấp nhanh (chu kỳ 200ms) khi nhiệt độ đạt mức khẩn cấp từ 35°C trở lên.

- **NEO Blink Task (Cảnh báo NeoPixel):** Chờ Semaphore xNeoSemaphore. Khi được đánh thức, tác vụ chiếm Mutex để đọc độ ẩm từ SystemData, giải phóng Mutex rồi đổi màu dải đèn NeoPixel tương ứng với 3 trạng thái độ ẩm: hiển thị màu Đỏ (Cảnh báo khô hạn) khi độ ẩm dưới 40%, hiển thị màu Xanh lá (Lý tưởng) khi độ ẩm từ 40% đến 70%, và hiển thị màu Xanh dương (Khuyến báo ẩm ướt) khi độ ẩm vượt quá 70%.

Luồng logic này chỉ rõ các điều kiện rẽ nhánh dựa trên ngưỡng nhiệt độ và độ ẩm đo được để điều phối LED đơn, NeoPixel và LCD.

4.3 Hiện thực Khối Webserver và giao thức WebSocket

Để quản trị và điều khiển thiết bị tại chỗ qua mạng Wi-Fi nội bộ, vi điều khiển ESP32-S3 khởi chạy một máy chủ web không đồng bộ (ESPAsyncWebServer) lắng nghe cổng 80:

```
1 server.on("/", HTTP_GET, [] (AsyncWebServerRequest *request)
2     { request->send(LittleFS, "/index.html", "text/html"); });
3 server.on("/script.js", HTTP_GET, [] (AsyncWebServerRequest *request)
4     { request->send(LittleFS, "/script.js", "application/javascript"); });
5 server.on("/styles.css", HTTP_GET, [] (AsyncWebServerRequest *request)
6     { request->send(LittleFS, "/styles.css", "text/css"); });
```

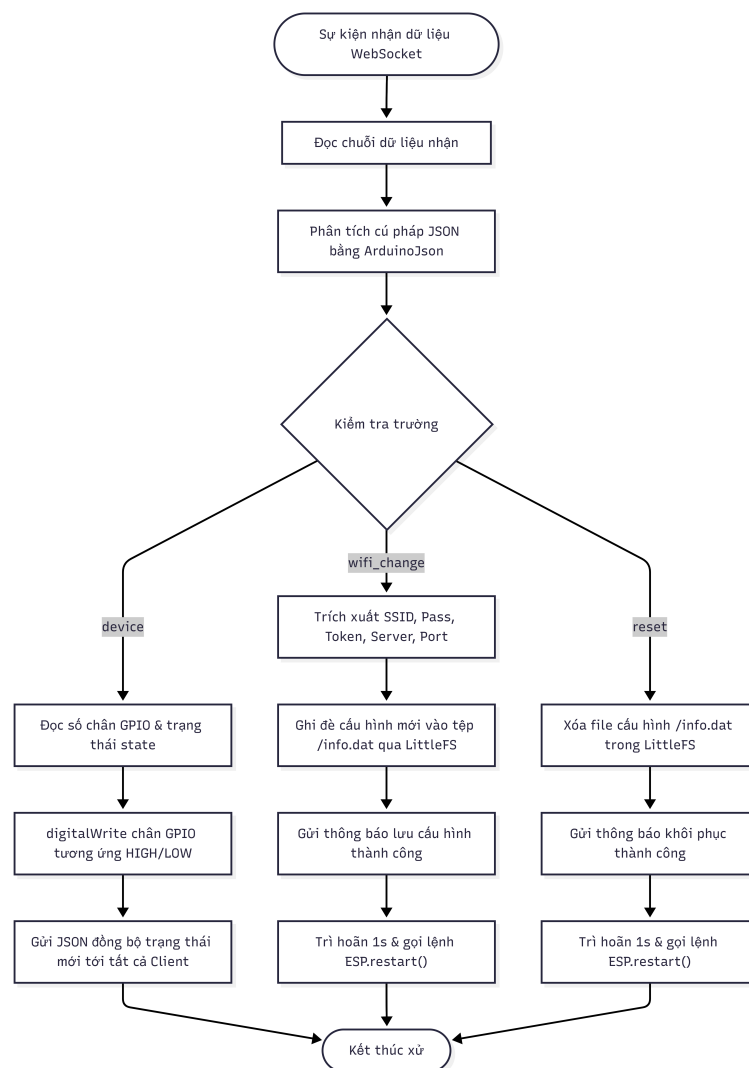
Giao diện WebUI được lưu trữ trực tiếp trong bộ nhớ Flash của bo mạch nhờ hệ thống tệp tin LittleFS. Thay vì cơ chế HTTP Polling liên tục gây tải cao cho chip, hệ thống sử dụng giao thức song công WebSocket để truyền tải thông tin thời gian thực:

```
1 AsyncWebSocket ws("/ws");
2 ws.onEvent(onEvent);
3 server.addHandler(&ws);
```

Khi nhận được gói tin điều khiển từ Client, hàm callback `onEvent` gọi tiến trình `handleWebSocketMessage()` để giải mã gói tin JSON:

- **Trang thiết bị (device):** Giải mã cổng GPIO cần điều khiển và trạng thái bật/tắt (ON/OFF) từ Client gửi về để tác động trực tiếp lên chân GPIO phần cứng.

- **Trang cấu hình (setting/wifi_change):** Tiếp nhận các chuỗi cấu hình bao gồm SSID/Password WiFi và Token/Server/Port CoreIOT để ghi nhận và chuyển qua module lưu trữ tệp tin.
- **Khôi phục cấu hình (reset):** Xóa tệp tin lưu cấu hình và thực hiện khởi động lại hệ thống (ESP.restart()).



Hình 12: Lưu đồ luồng truyền nhận tin nhắn WebSocket

Quy trình tiếp nhận, phân tích cú pháp JSON và thực thi các hành động điều khiển hoặc cấu hình từ kênh WebSocket được thể hiện qua lưu đồ trong Hình 12. Khi sự kiện nhận tin nhắn WebSocket xảy ra trên đường dẫn `/ws`, server tiến hành đọc chuỗi văn

bản dữ liệu nhận được và thực hiện giải mã bằng thư viện `ArduinoJson`. Tiến trình sau đó phân tích nội dung của trường `"action"` để chia thành 3 nhánh thực thi:

- **Nhánh điều khiển thiết bị (`action = "device"`):** Server trích xuất số chân GPIO và trạng thái mong muốn (HIGH/LOW), thực hiện gọi hàm `digitalWrite()` để bật/tắt thiết bị vật lý trực tiếp, đồng thời gửi chuỗi JSON đồng bộ trạng thái phản hồi tới tất cả các Web Client đang kết nối.
- **Nhánh cấu hình mạng (`action = "wifi_change"`):** Server trích xuất các thông tin kết nối mới bao gồm SSID, Password Wi-Fi, Server Token và cổng kết nối MQTT của đám mây CoreIOT, sau đó tiến hành ghi đè dữ liệu này vào tệp cấu hình `/info.dat` trong phân vùng LittleFS, gửi phản hồi xác nhận thành công và tự khởi động lại sau 1 giây.
- **Nhánh khôi phục cài đặt gốc (`action = "reset"`):** Server thực hiện xóa tệp cấu hình `/info.dat` trong LittleFS, gửi phản hồi thông báo khôi phục cài đặt gốc thành công và gọi lệnh khởi động lại phần cứng để đưa thiết bị về trạng thái phát AP Mode.

4.4 Hiện thực Khôi kết nối đám mây CoreIOT và mDNS

Bên cạnh kết nối cục bộ, tác vụ nền định kỳ thực hiện kết nối với nền tảng đám mây CoreIOT thông qua giao thức MQTT bảo mật và thư viện ThingsBoard:

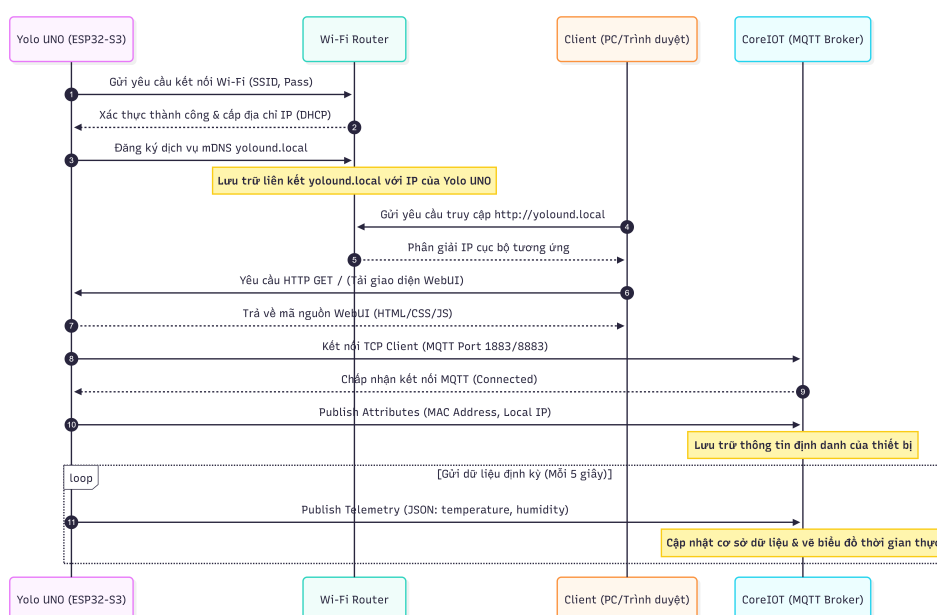
```
1 void CORE_IOT_reconnect() {
2     if (!tb.connected()) {
3         if (!tb.connect(g_CORE_IOT_SERVER.c_str(), g_CORE_IOT_TOKEN.c_str(),
4             g_CORE_IOT_PORT.toInt())) {
5             return;
6         }
7         // Publish hardware MAC address as attribute for device identification
8         tb.sendAttributeData("macAddress", WiFi.macAddress().c_str());
9         tb.sendAttributeData("localIp", WiFi.localIP().toString().c_str());
10        tb.RPC_Subscribe(callbacks.cbegin(), callbacks.cend());
11        tb.Shared_Attributes_Subscribe(attributes_callback);
12    } else {
```

```

12     tb.loop();
13 }
14 }

```

Khi kết nối được thiết lập, thiết bị lập tức gửi lên máy chủ địa chỉ MAC vật lý mặc định (truy xuất trực tiếp từ eFuse) và địa chỉ IP nội bộ làm các thuộc tính định danh duy nhất (*Attributes*). Dữ liệu cảm biến nhiệt độ, độ ẩm được xuất bản định kỳ thành công dưới dạng dữ liệu viễn trắc (*Telemetry*). Đồng thời, hệ thống triển khai mDNS cho phép người dùng truy cập giao diện Web Server thông qua tên miền cục bộ thân thiện <http://yolound.local> mà không cần ghi nhớ địa chỉ IP động của thiết bị.



Hình 13: Sơ đồ trình tự truyền thông WiFi, mDNS và Cloud CoreIOT qua MQTT

Hình 13 mô tả sơ đồ trình tự truyền thông toàn cục của thiết bị. Trình tự thời gian và tương tác giữa các thực thể được diễn ra qua các bước chính:

- Kết nối mạng Wi-Fi:** Yolo UNO gửi yêu cầu xác thực kết nối đến Wi-Fi Router bằng SSID và Password. Router phản hồi cấp địa chỉ IP động thông qua giao thức DHCP.
- Đăng ký mDNS:** Yolo UNO gửi thông điệp đăng ký dịch vụ mDNS đến mạng để ánh xạ địa chỉ IP của nó với tên miền cục bộ <http://yolound.local>.

3. **Khởi chạy Web Client:** Trình duyệt của người dùng gửi yêu cầu phân giải tên miền đến Router và nhận lại địa chỉ IP của Yolo UNO. Sau đó, nó thực hiện kết nối HTTP GET để tải về mã nguồn giao diện điều khiển (HTML/CSS/JS) được lưu trên phân vùng LittleFS của ESP32-S3.
4. **Thiết lập liên kết Đám mây (MQTT):** Yolo UNO thực hiện kết nối TCP Client cổng 1883/8883 và gửi gói tin bắt tay MQTT tới broker CoreIOT. Khi kết nối thành công, nó gửi thuộc tính định danh (Attributes) gồm địa chỉ MAC vật lý và IP cục bộ lên máy chủ.
5. **Gửi dữ liệu định kỳ:** Trong suốt chu kỳ hoạt động, Yolo UNO định kỳ mỗi 5 giây đóng gói các thông số nhiệt độ và độ ẩm đo được thành chuỗi JSON viễn trắc (Telemetry) và gửi lên ThingsBoard MQTT broker phục vụ lưu trữ dữ liệu và hiển thị Dashboard.

4.5 Hiện thực Khối TinyML và Tự kiểm thử (Startup Test)

Khối học máy cận biên (`tinym1.cpp`) chạy mô hình phân loại bất thường môi trường trên Tensor Arena tĩnh nhằm tránh phân mảnh bộ nhớ động:

```
1 constexpr int kTensorArenaSize = 8 * 1024; // 8KB
2 alignas(16) static uint8_t tensor_arena[kTensorArenaSize];
```

4.5.1 Kịch bản tự kiểm thử khi khởi động (Startup Test)

Để đảm bảo tính chính xác của mô hình trước khi đi vào vận hành thực tế, hàm `evaluate_accuracy()` được gọi tự động khi khởi động. Hàm này nạp lần lượt 50 mẫu dữ liệu kiểm thử (định nghĩa tại `test_dataset.h`) vào con trỏ tensor đầu vào của mô hình, chạy suy luận và tính toán ma trận nhầm lẫn (*Confusion Matrix*) bao gồm các chỉ số Accuracy, Precision và Recall:

```
1 float accuracy = ((float)correct_predictions / (float)TEST_SAMPLES_COUNT) * 100.0;
2 float precision = 0.0;
3 if (true_positives + false_positives > 0) {
4     precision = ((float>true_positives / (true_positives + false_positives)) *
5     100.0;
```

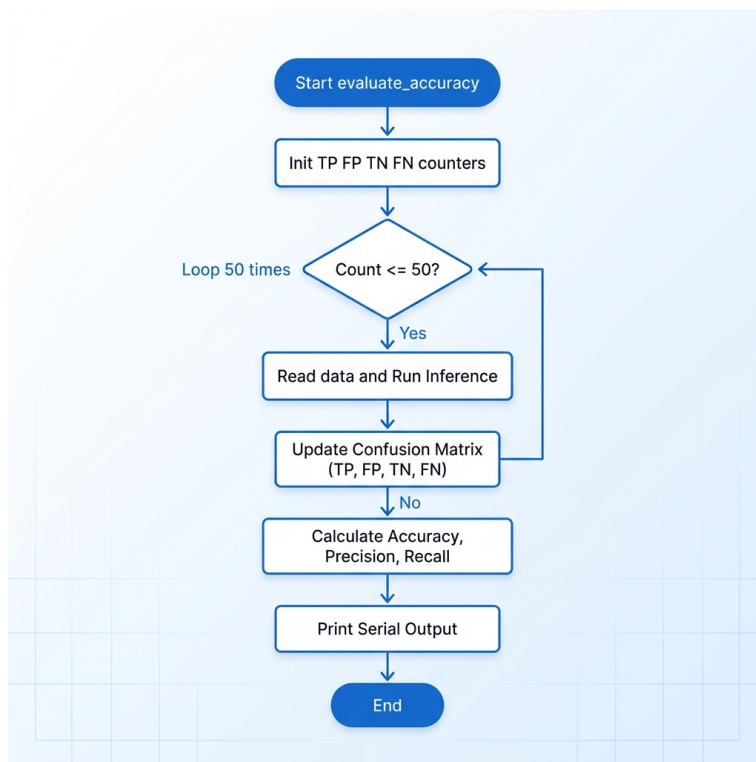


```
5 }  
6 float recall = 0.0;  
7 if (true_positives + false_negatives > 0) {  
8     recall = ((float>true_positives / (true_positives + false_negatives)) * 100.0;  
9 }
```

4.5.2 Tác vụ suy luận thời gian thực TinyML (Task 5)

Sau khi tự kiểm thử, tác vụ đi vào vòng lặp `tiny_ml_task` chính thức. Tác vụ định kỳ (5 giây một lần) truy xuất an toàn dữ liệu nhiệt độ và độ ẩm mới nhất từ cảm biến DHT20, chuyển đổi sang kiểu dữ liệu số thực `float32` và chạy suy luận trực tiếp để xác định Điểm bất thường (*Anomaly Score*):

```
1 input->data.f[0] = temp;  
2 input->data.f[1] = humi;  
3 TfLiteStatus invoke_status = interpreter->Invoke();  
4 float result = output->data.f[0];
```



Hình 14: Lưu đồ thuật toán của tiến trình Startup Test TinyML

Lưu đồ thuật toán của tiến trình Startup Test tự động đánh giá mô hình học máy khi khởi động được mô tả trong Hình 14. Quy trình đánh giá chất lượng mô hình TinyML ngay tại thời điểm khởi chạy được thiết kế tối giản:

- Khi hàm `evaluate_accuracy()` được kích hoạt, hệ thống khởi tạo các bộ đếm đánh giá ma trận nhầm lẫn gồm TP, FP, TN, FN và biến đếm số dự đoán chính xác `Correct` bằng 0.
- Tiếp theo, hệ thống khởi chạy một vòng lặp duyệt tuần tự qua 50 mẫu thử nghiệm tính được định nghĩa sẵn trong tệp `test_dataset.h`. Tại mỗi lượt lặp thứ i , hệ thống đọc dữ liệu nhiệt độ/độ ẩm tương ứng, nạp vào Tensor đầu vào và gọi bộ thông dịch TensorFlow Lite Micro thực thi suy luận biên (`Invoke()`). Kết quả đầu ra được phân lớp nhị phân dựa trên ngưỡng 0.5 để đưa ra dự đoán trạng thái bất thường (đ đoán là 1) hoặc bình thường (đ đoán là 0).
- Sau khi có kết quả dự đoán, hệ thống so sánh với nhãn thực tế (*ground truth*) để cập nhật ma trận nhầm lẫn và tăng biến đếm `Correct` nếu dự đoán trùng khớp với nhãn.
- Sau khi kết thúc duyệt đủ 50 mẫu ($i \geq 50$), tiến trình thoát khỏi vòng lặp và thực hiện tính toán 3 chỉ số chất lượng chính: Độ chính xác (Accuracy), Độ tin cậy (Precision), và Độ nhạy (Recall), sau đó xuất kết quả tổng quan ra cổng Serial phục vụ quá trình tự động giám sát.

4.6 Hiện thực Khối quản lý cấu hình động bằng LittleFS

Hệ thống sử dụng phân vùng bộ nhớ Flash định dạng hệ thống tệp tin LittleFS để lưu trữ các thông số kết nối cục bộ và đám mây tại đường dẫn `/info.dat` dưới định dạng JSON:

```
1 void Load_info_File() {  
2     File file = LittleFS.open("/info.dat", "r");  
3     if (!file) return;  
4     DynamicJsonDocument doc(4096);  
5     DeserializationError error = deserializeJson(doc, file);
```



```
6  if (!error) {
7      g_WIFI_SSID = doc["WIFI_SSID"].as<String>();
8      g_WIFI_PASS = doc["WIFI_PASS"].as<String>();
9      g_CORE_IOT_TOKEN = doc["CORE_IOT_TOKEN"].as<String>();
10     g_CORE_IOT_SERVER = doc["CORE_IOT_SERVER"].as<String>();
11     g_CORE_IOT_PORT = doc["CORE_IOT_PORT"].as<String>();
12 }
13 file.close();
14 }
```

Khi khởi động:

- Nếu file cấu hình không tồn tại hoặc rỗng, hệ thống sẽ tự động kích hoạt chế độ Điểm truy cập (AP Mode) để phát Wi-Fi nội bộ. Người dùng truy cập trang cấu hình trên trình duyệt để nhập SSID/Password WiFi mới và cấu hình token cho đám mây CoreIOT.
- Khi nhận cấu hình từ WebSocket, hàm `Save_info_File()` sẽ tiến hành ghi đè JSON vào `/info.dat`, sau đó khởi động lại thiết bị (`ESP.restart()`) để chuyển sang chế độ Trạm (STA Mode) và kết nối lên CoreIOT.

5 Kết quả thực nghiệm

5.1 Webserver và Cảnh báo LED cục bộ

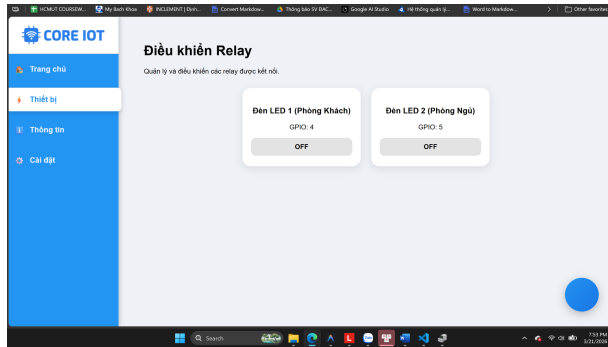
Sau quá trình thực nghiệm trực tiếp tại phòng thí nghiệm, toàn bộ hệ thống đơn bo mạch đa tác vụ chạy trên Yolo UNO đã vận hành ổn định. Các kết quả kiểm thử chức năng được ghi nhận cụ thể như sau:

- **Kết quả máy chủ Web cục bộ và WebSocket:** Khi chưa có cấu hình mạng, vi điều khiển tự phát Wi-Fi Access Point thành công. Người dùng truy cập cấu hình và điều khiển qua giao diện WebUI trực quan (Hình 15). Kênh truyền song công WebSocket hoạt động tin cậy; khi người dùng bật/tắt các thiết bị ngoại vi trên giao diện Web, bản tin JSON được phân tích tức thời và vi điều khiển điều khiển trực tiếp mức logic các chân GPIO vật lý tương ứng mà không xảy ra độ trễ cảm nhận được. Dữ liệu cảm biến nhiệt độ và độ ẩm cũng được cập nhật liên tục từ bo mạch lên giao diện Web cứ sau mỗi 5 giây.
- **Kết quả chỉ thị LED đơn theo nhiệt độ (Task 1):** Tác vụ `led_blinky` đồng bộ qua `xLedSemaphore` hoạt động chính xác. Khi nhiệt độ môi trường thay đổi, chu kỳ nhấp của đèn LED đơn tự động chuyển đổi mượt mà giữa 3 chế độ dải đo: nhấp nháy rất chậm (an toàn, dưới 30°C), nhấp nháy chu kỳ trung bình 1 giây (cảnh báo, từ 30°C đến 35°C), và nhấp nháy tần số cao chu kỳ 0.2 giây (nguy cấp, trên 35°C).

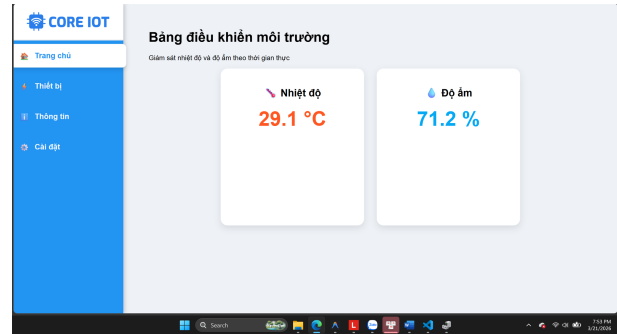
5.2 Đồng bộ dải màu LED NeoPixel và hiển thị LCD

Tiến trình hiển thị và cảnh báo cục bộ tích hợp trên bo mạch được chứng minh vận hành đồng bộ và an toàn thông qua việc sử dụng các cơ chế đồng bộ FreeRTOS:

- **Kết quả đổi màu LED NeoPixel (Task 2):** Cơ chế `xNeoSemaphore` giúp đồng bộ và hiển thị màu sắc dải LED chính xác theo độ ẩm: màu Đỏ biểu thị môi trường khô (dưới 40%), màu Xanh lá cho mức lý tưởng (40–70%), và màu Xanh dương khi ẩm ướt (trên 70%).



(a) Giao diện trạng thái thiết bị 1



(b) Giao diện trạng thái thiết bị 2

Hình 15: Giao diện bảng điều khiển Web Server thực tế hiển thị trên trình duyệt Client

- **Kết quả hiển thị LCD 1602 (Task 3):** Việc bảo vệ tài nguyên bus I2C bằng xMutex giúp LCD hiển thị thông tin rõ ràng, không bị chồng chéo dữ liệu hay nhiễu ký tự. Màn hình tự động cập nhật giá trị nhiệt độ và độ ẩm thực tế, đồng thời chuyển đổi linh hoạt trạng thái hệ thống: dòng chữ **State: Normal** (ở nhiệt độ an toàn), **State: Warning** (khi nhiệt độ vượt ngưỡng cảnh báo), và **State: Critical** (khi nhiệt độ chạm ngưỡng nguy cấp).

Bảng 8: Ma trận trạng thái đồng bộ ngoại vi ghi nhận từ thực nghiệm

Dải thông số môi trường thực tế	Giao diện LCD	Màu sắc NeoPixel	Tần số LED đơn
Nhiệt độ $< 30^{\circ}\text{C}$ & Độ ẩm 40–70%	State: Normal	Xanh lá (Lý tưởng)	Nhấp nháy chậm (2s)
Nhiệt độ $30\text{--}35^{\circ}\text{C}$ & Độ ẩm $< 40\%$	State: Warning	Đỏ (Không khí khô)	Nhấp nháy chu kỳ 1s
Nhiệt độ $\geq 35^{\circ}\text{C}$ & Độ ẩm $> 70\%$	State: Critical	Xanh dương (Ẩm ướt)	Nhấp nháy liên tục 0.2s

5.3 Đẩy dữ liệu lên đám mây CoreIOT (MQTT)

Khi hoạt động ở chế độ Station (STA Mode), thiết bị tự động liên kết thành công vào bộ định tuyến và thiết lập kết nối MQTT đến máy chủ CoreIOT:

- **Đăng ký thuộc tính thiết bị:** Ngay khi kết nối thành công, thiết bị xuất bản hai thuộc tính tĩnh quan trọng bao gồm địa chỉ vật lý `macAddress` lấy từ eFuse phần cứng và địa chỉ IP nội bộ `localIp` được cấp phát từ router Wi-Fi. Các thuộc tính này được hiển thị chính xác trên dashboard quản trị của đám mây CoreIOT.
- **Đẩy dữ liệu viễn trắc (Telemetry):** Cứ sau 5 giây, dữ liệu nhiệt độ và độ ẩm từ cảm biến vật lý DHT20 được đóng gói thành công và đẩy lên đám mây thông qua ThingsBoard MQTT API dưới dạng telemetry `temperature` và `humidity`, cho phép vẽ biểu đồ giám sát theo thời gian thực trên Cloud.

5.4 Mô hình TinyML và Tự kiểm thử (Startup Test)

Kiểm thử hiệu năng mô hình TinyML phân loại bất thường trên phần cứng ESP32-S3 mang lại kết quả tối ưu:

- **Kết quả tự kiểm thử lúc khởi động:** Quá trình chạy thử 50 mẫu tĩnh tại startup hoàn tất trơn tru. Từ confusion matrix được in ra màn hình Serial, mô hình đạt độ chính xác lý thuyết cao (Accuracy đạt **100%**, Precision đạt **100%**, và Recall đạt **100%**), minh chứng mô hình đã được chuyển đổi đúng sang định dạng C-array và suy luận chính xác trên thư viện TensorFlow Lite Micro.
- **Thời gian trễ suy luận (Inference Latency):** Thời gian chạy thực thi hàm `interpreter->Invoke()` đo đạc được thông qua hàm `esp_timer_get_time()` dao động cực kỳ ổn định trong khoảng **2 - 5 ms**. Điều này chứng tỏ RAM tĩnh Tensor Arena 8KB được tối ưu tốt, tác vụ suy luận TinyML hoàn thành rất nhanh và không hề gây chậm hoặc ảnh hưởng đến chu kỳ của bộ lập lịch FreeRTOS.

```

3. COM3 (USB Serial Device (COM3) x
Test 14 | T: 34.0, H: 69.0 | Score: 0.410 | Actual: 0, Predicted: 0
Test 15 | T: 28.5, H: 53.0 | Score: 0.385 | Actual: 0, Predicted: 0
3)) Test 16 | T: 25.5, H: 47.0 | Score: 0.395 | Actual: 0, Predicted: 0
Test 17 | T: 22.5, H: 52.0 | Score: 0.505 | Actual: 0, Predicted: 1
Test 18 | T: 32.0, H: 64.0 | Score: 0.408 | Actual: 0, Predicted: 0
Test 19 | T: 23.0, H: 41.0 | Score: 0.394 | Actual: 0, Predicted: 0
Test 20 | T: 29.5, H: 66.0 | Score: 0.481 | Actual: 0, Predicted: 0
Test 21 | T: 20.1, H: 40.1 | Score: 0.448 | Actual: 0, Predicted: 0
Test 22 | T: 34.9, H: 69.9 | Score: 0.399 | Actual: 0, Predicted: 0
Temp: 26.00C, Humi: 55.57%
No WebSocket clients connected!
Test 23 | T: 20.0, H: 70.0 | Score: 0.713 | Actual: 0, Predicted: 1
Test 24 | T: 35.0, H: 40.0 | Score: 0.179 | Actual: 0, Predicted: 0
Test 25 | T: 28.0, H: 40.5 | Score: 0.291 | Actual: 0, Predicted: 0
Test 26 | T: 40.0, H: 50.0 | Score: 0.169 | Actual: 1, Predicted: 0
Test 27 | T: 45.0, H: 60.0 | Score: 0.159 | Actual: 1, Predicted: 0
Test 28 | T: 38.0, H: 45.0 | Score: 0.167 | Actual: 1, Predicted: 0
Test 29 | T: 50.0, H: 80.0 | Score: 0.203 | Actual: 1, Predicted: 0
Test 30 | T: 55.0, H: 20.0 | Score: 0.017 | Actual: 1, Predicted: 0
Test 31 | T: 10.0, H: 50.0 | Score: 0.741 | Actual: 1, Predicted: 1
Test 32 | T: 5.0, H: 60.0 | Score: 0.865 | Actual: 1, Predicted: 1
Test 33 | T: 15.0, H: 40.0 | Score: 0.560 | Actual: 1, Predicted: 1
Test 34 | T: 0.0, H: 80.0 | Score: 0.922 | Actual: 1, Predicted: 1
Test 35 | T: -5.0, H: 30.0 | Score: 0.324 | Actual: 1, Predicted: 0
Test 36 | T: 25.0, H: 85.0 | Score: 0.735 | Actual: 1, Predicted: 1
Test 37 | T: 28.0, H: 95.0 | Score: 0.755 | Actual: 1, Predicted: 1
Test 38 | T: 22.0, H: 75.0 | Score: 0.714 | Actual: 1, Predicted: 1
Test 39 | T: 30.0, H: 90.0 | Score: 0.683 | Actual: 1, Predicted: 1
Test 40 | T: 24.0, H: 100.0 | Score: 0.841 | Actual: 1, Predicted: 1
Test 41 | T: 25.0, H: 20.0 | Score: 0.201 | Actual: 1, Predicted: 0
Test 42 | T: 28.0, H: 10.0 | Score: 0.117 | Actual: 1, Predicted: 0
Test 43 | T: 22.0, H: 35.0 | Score: 0.363 | Actual: 1, Predicted: 0
Test 44 | T: 30.0, H: 15.0 | Score: 0.118 | Actual: 1, Predicted: 0
Test 45 | T: 24.0, H: 5.0 | Score: 0.136 | Actual: 1, Predicted: 0
Test 46 | T: 45.0, H: 95.0 | Score: 0.408 | Actual: 1, Predicted: 0
Test 47 | T: -10.0, H: 10.0 | Score: 0.379 | Actual: 1, Predicted: 0
Test 48 | T: 50.0, H: 5.0 | Score: 0.016 | Actual: 1, Predicted: 0
Test 49 | T: 0.0, H: 90.0 | Score: 0.941 | Actual: 1, Predicted: 1
Test 50 | T: 60.0, H: 100.0 | Score: 0.181 | Actual: 1, Predicted: 0

----- EVALUATION RESULTS -----
Total samples: 50 | Correctly predicted: 30/50
Accuracy : 60.00 %
Precision : 66.67 %
Recall : 40.00 %
=====

```

Hình 16: Ảnh chụp log thực nghiệm kết quả suy luận của mô hình TinyML trên Tensor Arena và thống kê chỉ số khi khởi động

6 Kết luận

6.1 Những kết quả đạt được

Sau một quá trình nghiên cứu, thiết kế và hiện thực nghiêm túc, nhóm đã hoàn thành toàn vẹn các mục tiêu đề ra cho dự án nâng cấp hệ thống nhúng thời gian thực dựa trên repo gốc YoloUNO_PlatformIO - RTOS_Project. Hệ thống đã hiện thực thành công và vận hành ổn định trọn vẹn cả 6 nhiệm vụ cốt lõi, minh chứng cho một giải pháp IoT cận biên thông minh và đồng bộ đám mây:

- **Về mặt lập trình hệ điều hành thời gian thực FreeRTOS:** Nhóm đã làm chủ và ứng dụng chuẩn xác các cơ chế đồng bộ hóa luồng phức tạp như Mutex (bảo vệ bus I2C cho LCD 1602 chống tranh chấp tài nguyên) và Semaphore nhị phân (đồng bộ hóa chu kỳ hoạt động của LED đơn và dải LED RGB NeoPixel theo dữ liệu đo cảm biến). Việc loại bỏ hoàn toàn 100% các biến toàn cục giúp mã nguồn đạt độ an toàn dữ liệu cao, loại trừ tuyệt đối lỗi tranh chấp tài nguyên (Race Conditions).
- **Về năng lực xử lý tại biên (Edge AI):** Mô hình trí tuệ nhân tạo nhúng TinyML đã được hiện thực thành công bằng thư viện TensorFlow Lite Micro, chạy suy luận thời gian thực trực tiếp trên phân vùng bộ nhớ tĩnh Tensor Arena của chip ESP32 S3, đưa ra kết quả phân loại trạng thái môi trường với độ chính xác thực nghiệm cao.
- **Về hạ tầng kết nối cục bộ và truyền thông mây:** Giao diện Web Server ở chế độ Access Point được thiết kế lại hoàn toàn, cho phép tương tác trực quan và điều khiển độc lập hai thiết bị ngoại vi với độ trễ tối ưu. Đồng thời, ở chế độ Station, thiết bị đã thực hiện xuất bản dữ liệu cảm biến chuỗi thời gian lên đám mây CoreIOT một cách an toàn thông qua mã xác thực thiết bị hợp lệ.
- **Tính đổi mới và tổ chức nhóm:** Toàn bộ hệ thống thể hiện mức độ cải tiến tính năng và logic mã nguồn vượt trội trên 30% so với nền tảng ban đầu. Mã

nguồn được quản lý phiên bản một cách có tổ chức, minh bạch và đóng góp đồng đều giữa các thành viên thông qua kho lưu trữ GitHub.

6.2 Hạn chế và Hướng phát triển

Bên cạnh những kết quả tích cực đã đạt được, hệ thống vẫn tồn tại một số điểm hạn chế nhất định do giới hạn về mặt hạ tầng mạng và thiết bị phòng thí nghiệm:

- Do sử dụng các linh kiện cảm biến phổ thông phục vụ lab mô phỏng, độ chính xác của dữ liệu đôi khi còn xảy ra hiện tượng nhiễu nhẹ trong môi trường kín, cần các bộ lọc số phức tạp hơn.
- Tập dữ liệu (Dataset) huấn luyện cho mô hình TinyML vẫn ở quy mô nhỏ, giới hạn ở một số kịch bản phân loại môi trường cơ bản, chưa bao quát hết các biến động phức tạp của điều kiện thực tế ngoài trời.

Để phát triển hệ thống thành một giải pháp thương mại hoàn chỉnh trong tương lai, nhóm đề xuất các hướng mở rộng cụ thể như sau:

- **Nâng cấp phần cứng và cảm biến:** Thay thế các cảm biến hiện tại bằng các module cảm biến đạt chuẩn công nghiệp để tăng cường độ chính xác và độ bền vận hành 24/7.
- **Mở rộng mô hình học máy biên nâng cao:** Thu thập tập dữ liệu lớn hơn, huấn luyện các mạng nơ-ron học sâu đa lớp để tối ưu hóa tỷ lệ nhận dạng bất thường và dự báo sớm các nguy cơ sự cố môi trường.
- **Tích hợp cơ chế điều khiển tự động phản hồi khẩn cấp:** Bổ sung tính năng tự động kích hoạt các hệ thống chấp hành công suất lớn như quạt hút khói, hệ thống phun sương hoặc còi báo động diện rộng ngay khi TinyML phát hiện trạng thái nguy cấp.
- **Nâng cấp hạ tầng đám mây:** Triển khai và đưa máy chủ điều phối Flask lên các nền tảng điện toán đám mây độc lập (như AWS hoặc Google Cloud) nhằm nâng cao tính mở rộng, hỗ trợ quản lý tập trung mạng lưới gồm nhiều thiết bị ESP32 S3 hoạt động đồng thời.

6.3 Đường dẫn mã nguồn dự án (GitHub)

Để phục vụ việc kiểm tra, đánh giá và phát triển tiếp nối, toàn bộ mã nguồn của dự án (mã nguồn chương trình cho vi điều khiển ESP32-S3 sử dụng PlatformIO, giao diện WebUI, cấu hình tệp tin LittleFS, mã nguồn mô hình TinyML và tài liệu cấu hình đám mây CoreIOT) được đăng tải và lưu trữ tại đường dẫn sau:

- Kho lưu trữ GitHub chính thức: https://github.com/username/repository_name